Podręcznik wprowadzający w tematykę przedmiotu

Strona: LeIA 2026
Kurs: Zaawansowane aplikacje internetowe (2026Z)
Książka: Podręcznik wprowadzający w tematykę przedmiotu

Wydrukowane przez użytkownika: Tymon Kwiatkowski
Data: piątek, 2 października 2026, 12:07

Opis

Spis treści

1. Strona tytułowa

2. Czym są zaawansowane aplikacje internetowe, czyli wstęp lekko historyczny
2.1. Dawno, dawno temu...
2.2. Automatyzacja generowania dokumentów HTML
2.3. Kontekst telekomunikacyjny
2.4. Wzrost znaczenia języka JavaScript
2.5. Współczesne aplikacje internetowe
2.6. Mobilne aplikacje internetowe
2.7. Zaawansowane aplikacje internetowe

3. Podstawowy podział na warstwy: frontend i backend
3.1. Frontend
3.2. Backend
3.3. Nieostra granica backend-frontend
3.4. SSR – Server Side Rendering
3.5. CSR – Client Side Rendering
3.6. Klient

4. Architektury aplikacji internetowych
4.1. Architektura klient - serwer
4.2. Architektura monolityczna
4.3. Mikroserwisy

5. Wzorce projektowe
5.1. BBOM - Big Ball of Mud
5.2. MVC - Model View Controller
5.3. MVVM - Model View ViewModel

6. Protokoły komunikacyjne
6.1. HTTP
6.2. WebSocket

7. RESTful API
7.1. Koncepcja zasobu
7.2. Zasady identyfikowania zasobów
7.3. Metody HTTP
7.4. Specyfikacja OpenAPI

1. Strona tytułowa

Niniejszy moduł poświęcony jest podstawowym koncepcjom i narzędziom wykorzystywanym przy tworzeniu aplikacji internetowych.

Słowa kluczowe: aplikacje internetowe, programowanie, *backend*, serwer, REST API, narzędzia deweloperskie

Wymagana wiedza bazowa: podstawy programowania aplikacji internetowych, podstawowa znajomość języka JavaScript oraz HTML i CSS

Autor: dr inż. Przemysław Korpas, Instytut Radioelektroniki i Technik Multimedialnych

2. Czym są zaawansowane aplikacje internetowe, czyli wstęp lekko historyczny

Chcąc odpowiedzieć na pytanie zawarte w tytule, należałoby zastanowić się nad tym, czym aplikacje internetowe są w ogóle, co sprawia, że możemy określić je mianem „zaawansowane” i co tak właściwie rozumiemy pod hasłem Internet.

2.1. Dawno, dawno temu...

Historycznie rzecz ujmując, początki przybliżania Internetu (jako medium transmisyjnego) dla powszechnego odbiorcy datuje się na rok 1991, kiedy to na University of Minnesota opracowany został protokół *Gopher* wraz z odpowiednim oprogramowaniem serwerowym i klienckim. System pozwalał na udostępnianie polinkowanych ze sobą treści, czyli de facto stanowił protoplastę stron internetowych. Zaprojektowany był z uwzględnieniem występujących ówcześnie ograniczeń technicznych – tak w zakresie prędkości przesyłania danych za pomocą modemów, jak i możliwości komputerów – to znaczy zorientowany był przede wszystkim na przekazywanie treści w formie tekstowej. Mimo to, zdobył ogromną popularność – dość powiedzieć, że w roku 1994 funkcjonowało 6958 serwerów Gophera. Był to jednocześnie szczyt popularności tego systemu jak i początek jej nagłego schyłku, gdyż uwaga społeczności masowo skierowała się w kierunku innej technologii internetowej – World Wide Web.

Popularne dziś „www" autorstwa Tima Bernersa-Lee, zadebiutowało również w 1991 roku w CERN w Szwajcarii. Podobnie jak Gopher, klient WWW (który obecnie określamy pojęciem przeglądarki internetowej) mógł pobierać zdecentralizowane zasoby internetowe z całego świata. Jednak w przeciwieństwie do Gophera, WWW wykorzystywało model skoncentrowany na dokumentach. Zamiast hierarchicznego menu, każdy serwer dostarczał zbiór dokumentów tekstowych z hipertekstowymi linkami do ich wzajemnego wiązania ze sobą. Był to kolejny krok w kierunku decentralizacji dystrybucji dokumentów i plików. Jednakże użyteczność stron internetowych pozwalających na osadzanie w dokumentach multimediów, nie od razu była oczywista w czasach, gdy obsługa komputerów (a zatem i łączność z siecią) odbywała się za pomocą tekstowych interfejsów użytkownika. Tu warto wspomnieć o tekstowej przeglądarce internetowej Lynx. Dopiero gdy w 1993 roku wydana została przeglądarka Mosaic, będąca pierwszą przeglądarką z wbudowaną obsługą grafiki, a pod strzechy stopniowo zaczęły trafiać komputery z systemem operacyjnym Windows, sytuacja uległa znaczącej zmianie. Na kres popularności Gophera niewątpliwie wpłynęła także zapowiedź wprowadzenia opłat licencyjnych dla użytkowników komercyjnych.

Więcej szczegółów znajdziesz tutaj:

- Michael Grobe - An Early History of Lynx - wspomnienia jednego z twórców tekstowej przeglądarki Lynx,
- Benj Edwards - The Web Before the Web: A Look Back at Gopher - artykuł opisujący historię systemu Gopher.

2.2. Automatyzacja generowania dokumentów HTML

Strony www w początkowym okresie budowane były jako zbiory polinkowanych ze sobą statycznych dokumentów HTML – czyli literalnie plików wgranych na serwer. Dość szybko okazało się, że pliki HTML w ramach jednego serwisu zawierają powtarzające się w wielu miejscach identyczne fragmenty kodu, np. nagłówki, stopki, menu, itp. Ręczne przygotowywanie dużej liczby podobnych do siebie plików HTML było i jest niepraktyczne, w związku z czym opracowano narzędzia pozwalające automatyzować po stronie serwera generowanie dokumentów HTML wysyłanych do przeglądarki. Jedną z dróg było wykorzystanie interfejsu CGI (ang. *Common Gateway Interface*), który unifikował sposób przekazywania żądań przesyłanych od klienta (tj. przeglądarki internetowej) poprzez serwer HTTP, do programu generującego odpowiedź. W takim przypadku mówimy o tzw. *server-side rendering*, czyli generowaniu kompletnego dokumentu HTML po stronie serwera w odpowiedzi na żądanie. Rolę takiego programu pełniły często skrypty w języku Perl lub inne dedykowane oprogramowanie, które mogło być napisane nawet w języku C czy C++. Technologia CGI obecnie uważana jest za przestarzałą i niewydajną z uwagi na tworzenie nowego procesu za każdym razem, gdy przetwarzano kolejne żądanie.

Więcej o CGI przeczytasz tutaj na portalu Geeks for Geeks (choć... traktuj to zdecydowanie jako ciekawostkę niż technikę, z której dziś skorzystasz).

Jako narzędzie do automatyzacji generowania stron sprawdził się stworzony w 1994 roku język *Personal Home Page Toolkits*, dziś znany powszechnie jako **PHP**. Początkowo implementowany był właśnie jako rozszerzenie CGI do serwera HTTP. Wśród twórców internetowych zdobył on ogromną popularność. Kilka lat później pojawiły się technologie, które wykorzystywały do implementacji dynamicznych stron internetowych język C# (**Active Server Pages, ASP**, rok 1998) czy język Java (**Java Server Pages, JSP**, rok 1999) – one również znalazły swoje nisze i stopniowo zdobywały popularność. W późniejszych latach do popularnych rozwiązań backendowych dołączyły te wykorzystujące język **Python** (np. stosunkowo prosta biblioteka Flask, czy też rozbudowany framework Django), Ruby z frameworkiem Ruby on Rails, czy JavaScript ze środowiskiem Node.js.

Realizacja serwisów internetowych wymagała także wygodnego sposobu przechowywania danych – tutaj z pomocą przyszły technologie **relacyjnych baz danych** obsługujących język zapytań SQL takie jak MySQL (rok 1996) czy PostgreSQL (1996, wcześniej występujący pod nazwą Postgres95).

W wolnej chwili przy kawie możesz zerknąć także do tych artykułów:

- History of PHP
- Sasha Pachev, Understanding MySQL Internals, O'Reilly Media, 2007 - Chapter 1. MySQL History and Architecture
- A Brief History of PostgreSQL

2.3. Kontekst telekomunikacyjny

Automatyczne generowanie dokumentów HTML pozwalało już budować całkiem skomplikowane serwisy, które z dzisiejszej perspektywy moglibyśmy nazwać pierwszymi aplikacjami internetowymi. Jednakże koncepcja przesyłania z serwera do klienta pełnej treści dokumentu HTML po każdym kliknięciu w aplikacji była problematyczna o tyle, że samo przekazywanie danych poprzez sieć trwało pewien czas – na tyle długi, że często był on przeszkadzający w swobodnym i płynnym korzystaniu z aplikacji. Przesłanie prostej strony trwało ułamki sekundy, ale przy bardziej złożonych dokumentach czas oczekiwania mógł wynosić nawet i kilka-kilkanaście sekund – w zależności od parametrów łącza oraz rozmiaru dokumentu.

Tu należy nadmienić, że wraz z rozwojem technologii telekomunikacyjnych, na przestrzeni lat systematycznie wzrastała średnia przepustowość łącz wykorzystywanych do realizacji dostępu do Internetu. Na początku lat 90-tych XX wieku dostęp do ogólnoświatowej sieci możliwy był w zasadzie wyłącznie za pośrednictwem tzw. połączeń wdzwanianych (ang. *dial-up*) z prędkościami rzędu 14.4 kbps. Jeszcze na przełomie XX i XXI wieku pojawiły się modemy *dial-up* 56 kbps w standardzie V.92/V.90. W kolejnej dekadzie (do 1. stycznia 2011) Telekomunikacja Polska S.A. (dominujący operator usług telekomunikacyjnych w Polsce) oferował także usługę o nazwie Stały Dostęp do Internetu (SDI) pozwalającą na dostęp do sieci z prędkością 115.2 kbps - zdecydowaną zaletą była stała opłata, niezależna od czasu trwania połączenia. W międzyczasie wdrożone zostały technologie ADSL, później VDSL, pozwalające na klasycznej parze miedzianej uzyskać do 55 Mbps prędkości pobierania. Od kilku lat jesteśmy świadkami systematycznego przechodzenia z połączeń przewodowych miedzianych na światłowodowe (technologia *FTTH - Fiber to the Home*) lub na szerokopasmową łączność bezprzewodową za pośrednictwem sieci komórkowej LTE i 5G, a także łączność satelitarną (np. poprzez urządzenia Starlink). Obecnie (w roku 2024) dostępne są dla klientów indywidualnych łącza oferujące komunikację z prędkościami przekraczającymi nawet 1 Gbps.

Więcej o postępie technicznym w dostępie do sieci Internet przeczytasz w poniższych artykułach:

- Global average Internet speed, 1990-2050 (wraz z bogatą listą referencji),
- Sieć POLPAK,
- Stały Dostęp do Internetu,
- Neostrada.

Kontekst telekomunikacyjny jest istotny dla kierunku rozwoju aplikacji internetowych, gdyż parametry łącz dostępowych wykorzystywanych przez użytkowników końcowych stanowiły kiedyś główny czynnik wpływający na czas potrzebny do załadowania strony. Czas ten można wyrazić wzorem:

$t_{http} = t_{mdt_request} + t_{http_{pt}} + t_{mdt_response}$

gdzie:

- $t_{mdt_request}$ - czas przesłania pełnego żądania od klienta do serwera,
- t_{ps_http} - czas przetwarzania żądania HTTP przez serwer,
- $t_{mdt_response}$ - czas przesłania pełnej odpowiedz od serwera do klienta.

Twórca aplikacji internetowej ma bezpośredni wpływ na wartość czasu przetwarzania żądania HTTP. Wartość ta zależna jest bezpośrednio od rozwiązań przyjętych przez twórcę aplikacji sieciowej w zakresie obejmującym m.in. zagadnienia takie jak:

- złożoności oprogramowania uruchamianego w celu przetworzenia żądania (czyli właściwego kodu aplikacji),
- rodzaj zastosowanego serwera HTTP (np. Apache, nginx, Node.js),
- szybkości procesora zainstalowanego w serwerze,
- liczby żądań przetwarzanych równolegle w tym samym czasie przez serwer,
- wykorzystania cache po stronie serwera,
- wykorzystanie tzw. load balancera, rozdzielającego nadchodzące żądania pomiędzy wiele serwerów zajmujących się ich obsługą.

Z kolei czas niezbędny na przesłanie tych danych, tzw. czas dostarczenia komunikatu (ang. *message delivery time*, t_{mdt}) określić możemy w przybliżeniu za pomocą prostej relacji:

$t_{mdt} = \frac{MS*8}{NT}$

gdzie:

- **t_{mdt} - czas dostarczenia komunikatu** wyrażony w sekundach,
- **MS** - rozmiar danych do przesłania (ang. *message size*) wyrażony w bajtach (np. rozmiar odpowiedzi odsyłanej przez serwer, wraz z nagłówkami protokołu HTTP),
- **NT** - wypadkowa prędkość transmisji poprzez sieć (ang. *network throughput*) wyrażony w bitach na sekundę.

Przykład

Przyjmując **MS = 100 kB** w zależności od wypadkowej prędkości transmisji w sieci otrzymamy różne wyniki:

- dla **NT = 1 Mbps** uzyskujemy czas transmisji **t = 0.8 sekund**y,
- dla **NT = 100 Mbps** uzyskujemy czas transmisji **t** = **8 milisekund**.

Pierwszy przypadek reprezentuje łącze o kiepskich parametrach (np. sieć komórkową pracującą na skraju zasięgu lub w warunkach silnego przeciążenia), drugi przypadek może zachodzić np. w przypadku wykorzystywania łącza światłowodowego.

Choć twórca aplikacji internetowej nie ma wpływu na rodzaj łącza wykorzystywanego przez użytkownika końcowego, to przy projektowaniu powinien uwzględnić spodziewane jego parametry. Jeśli wiadomo, że aplikacja będzie wykorzystywana głównie na urządzeniach stacjonarnych, z szerokopasmowym dostępem do internetu, to jak widać na powyższym przykładzie przesłanie nawet 100 kB może nie wprowadzić istotnych opóźnień i tym samym można rozważyć pozostawienie sytuacji bez zmian. Jeśli jednak wiadomo, że specyfika aplikacji powoduje jej wykorzystanie głównie na telefonach komórkowych, projektant powinien rozważyć zmniejszenie ilości przesyłanych danych aby zminimalizować efekt zwiększonego czasu odpowiedzi aplikacji, np. poprzez:

- precyzyjną selekcję przesyłanych informacji (należy wybrać tylko te, które są niezbędne do zapewnienia funkcjonalności),
  - w szczególności: renderować widok po stronie klienta na podstawie przesłanych danych, nie przesyłać pełnego widoku np. w postaci dokumentów HTML,
- zastosowanie kompresji,
- w przypadku multimediów: wybór formatu lub nastaw kodeka skutkujących wyższym stopniem kompresji.

W przeszłości, gdy powszechnym było wykorzystywanie łącz o przepustowościach rzędu 50 - 100 kbps, szczególnie dbano o to, aby oszczędnie gospodarować dostępnymi zasobami i tym samym przesyłać poprzez sieć jak najmniej danych. Z pomocą przychodziły tutaj m.in. techniki dynamicznego renderowania zawartości strony po stronie klienta - o tym przeczytasz w kolejnym rozdziale.

Dla dociekliwych: od czego zależy wypadkowa prędkość transmisji?

Na wypadkową prędkość transmisji poprzez sieć wpływa kilka czynników. Dostępna przepustowość wykorzystywanego łącza dostępowego jest tylko jednym ze składników. Przyjrzyjmy się temu bliżej.

Aby wyznaczyć czas dostarczenia wiadomości (*ang. message delivery time*, t_{mdt}), należy uwzględnić następujące detale:

- **czas transmisji jednego pakietu** (ang. *packet transmission time*, t_{ptt}) - to czas od nadania pierwszego bitu komunikatu (pakietu) do nadania ostatniego bitu wiadomości. Czas transmisji pakietu w sekundach można uzyskać z rozmiaru pakietu w bitach (**PS**) i szybkości transmisji w bitach na sekundę (BR, ang. *bit-rate*) :
  $t_{ptt} = \frac{PS}{BR}$
  Przykładowo, dla sieci Ethernet 100 Mbps maksymalny rozmiar pakietu to 1526 bajtów, więc:
  $t_{ptt} = \frac{1526\ B * 8\ bit}{100 \cdot 10^6\ bps} = 122\ \mu s$
- **czas propagacji** (ang. *propagation delay*) - czas wynikający bezpośrednio z fizycznych właściwości wykorzystywanego medium transmisyjnego zgodnie z relacją:
  $t_{pd} = \frac{s}{v}$
  s - fizyczna odległość pomiędzy nadawcą a odbiorcą,
  v - prędkość propagacji fali elektromagnetycznej w wykorzystywanym medium (np. w parze miedzianej jest to około $2*10^8$ m/s, a dla fal radiowych $3*10^8$ m/s).
  Dla przykładu przyjmijmy, że transmisja odbywa się na odległość 10 km za pośrednictwem pary miedzianej. Czas propagacji wyniesie wtedy:
  $t_{pd} = \frac{10 \cdot 10^3\ m}{2 \cdot 10^8\ \frac{m}{s}} = 50\ \mu s$
- **czas dostarczenia pakietu** (ang. packet delivery time) - czas pomiędzy wysłaniem pierwszego bajtu przez nadawcę a odebraniem ostatniego bajtu przez odbiorcę stanowi sumę czasu transmisji oraz czasu propagacji i jest wyrażany wzorem:
  $t_{pdt} = t_{ptt} + t_{pd}$
  W praktyce przy transmisji poprzez sieć pakiety wędrują pomiędzy wieloma urządzeniami (np. routerami), więc każdy z odcinków łącz należy rozpatrywać osobno, a także dodać czas związany z kolejkowaniem i przetwarzaniem pakietów na urządzeniach pośredniczących. W sieciach rozległych czas ten jest rzędu milisekund.
- **czas round-trip** (ang. *round-trip time, t_{rtt}*) - czas pomiędzy rozpoczęciem transmisji przez nadawcę do momentu pełnego odebrania przez niego odpowiedzi. Na ten parametr wpływa zarówno czas propagacji t_{pr} (i to dwukrotnie, gdyż temu opóźnieniu podlega zarówno komunikat przesyłany do odbiorcy jak i odsyłana odpowiedź), jak i czas przetwarzania odpowiedzi (t_{ps}) - zgodnie ze wzorem:
  $t_{rtt} = 2\ \cdot t_{pd} + t_{ps}$
- **czas przetwarzania żądania** (ang. *processing delay*, t_{ps}) - czas niezbędny na przetworzenie pakietu przez odbiorcę (np. odesłanie potwierdzenia ACK). Czas ten zależny jest m.in. od chwilowego obciążenia węzła docelowego.
- **wypadkowa przepustowość** (ang. *network troughput*, *NT*) - dla wykorzystywanych przez protokół HTTP połączeń transportowych poprzez protokół TCP, z uwagi na zastosowany tam mechanizm kontroli przepływu (ang. *flow control*), wypadkową przepustowość można wyrazić jako stosunek rozmiaru okna (ang. window size, WS) do czasu round-trip:

$$NT = \frac{WS}{t_{rtt}}.$$

Więcej na ten temat przeczytasz m.in. w Wikipedii w artykule Transmission time.

2.4. Wzrost znaczenia języka JavaScript

Wracając do przeszłości, z jednej strony ograniczenia transmisyjne wymuszały niewielki rozmiar stron, ale z drugiej wskazywały na potrzebę manipulowanie zawartością prezentowanej strony internetowej po stronie przeglądarki w reakcji na działania użytkownika strony. Jedną z koncepcji było założenie, że komunikacja z serwerem (jako czasochłonna) powinna zachodzić wyłącznie w sytuacji absolutnej konieczności i ograniczać się do przesyłania wyłącznie niezbędnych danych (w miejsce przesyłania stron w całości renderowanych po stronie serwera).

Na tę okoliczność twórcy przeglądarki Netscape zaproponowali w 1995 roku osadzenie w ich przeglądarce interpretera nowegojęzyka programowania, jakim został JavaScript. JavaScript pozwolił na wprowadzanie manipulacji na elementach strony, tzw. w drzewie DOM (ang. *Document Object Model*) po załadowaniu treści strony z serwera. W rezultacie możliwe stało się m.in. dynamiczne osadzanie nowych treści na stronie, zmiana właściwości elementów, reagowanie na zdarzenia powodowane przez użytkownika (np. na kliknięcie, na ruch myszy nad danym elementem strony, itp.) a także na asynchroniczna komunikacja z serwerem. Ten ostatni element spopularyzował się przy wykorzystaniu koncepcji AJAX (ang. Asynchronous JavaScript and XML) dla tworzenia aplikacji internetowych, które wreszcie zaczęły działać relatywnie szybko. AJAX umożliwia wymianę z serwerem samych informacji (początkowo głównie w formie obiektów XML, później zdecydowanie częściej w formacie JSON), bez ich formatowania za pomocą kodu HTML, a sama budowa elementów drzewa DOM odbywa się jawnie za pomocą kodu JavaScript po stronie klienta.

Podejście to jest popularne również współcześnie, choć dynamizm budowania drzewa DOM można dziś określić mianem 100% - otóż korzystając ze współczesnych bibliotek i frameworków JavaScript takich jak np. React, Vue, AnguarJS, praktycznie cały interfejs użytkownika aplikacji może być (i w praktyce bardzo często jest) generowany dynamicznie po stronie klienta. Ponadto frameworki organizują kod części frontendowej aplikacji internetowych pod względem architektonicznym i zapewniają szereg narzędzi rozwiązujących typowe problemy związane m.in. z zarządzaniem stanem aplikacji, synchronizacją zmian elementów drzewa DOM itp.

Warto tutaj nadmienić, że w tzw. międzyczasie, na kilkanaście lat rozbłysła gwiazda technologii ożywiających strony internetowe za pomocą różnego rodzaju „wtyczek" pozwalających uruchamiać aplety Javy, multimedialne i atrakcyjne pod względem graficznym obiekty w technologiach Macromedia Flash i Microsoft Silverlight, czy nawet odtwarzacze multimedialne zainstalowane w systemie operacyjnym komputera, na którym działała przeglądarka. Było to dość karkołomne rozwiązanie z uwagi na konieczność instalacji dodatkowych wtyczek lub środowisk uruchomieniowych, a poza tym rodziło trudności z zapewnieniem bezpieczeństwa. Technologie Flash i Silverlight na szczęście odeszły w niepamięć wkrótce po upowszechnieniu się wsparcia dla nowoczesnych języków HTML5 oraz CSS3 w popularnych przeglądarkach (były to okolice roku 2015). Więcej na temat tej zmiany technologicznej przeczytasz w artykule: Jerry Smith - Moving to HTML5 Premium Media.

2.5. Współczesne aplikacje internetowe

Co ważne, współczesny język JavaScript wyposażony jest w szereg rozszerzeń, które umożliwiają m.in. wydajne (tj. z użyciem akceleracji sprzętowej) wyświetlanie grafiki 3D (WebGL API), manipulowanie dźwiękiem (Web Audio API), manipulowanie elementami strumieni wideo i dźwięku (WebCodecs API), wsparcie dla urządzeń wirtualnej rzeczywistości (WebXR Device API), geolokację (Geolocation API), dwukierunkową swobodną komunikację z serwerem (WebSockets API). Lista ta oczywiście nie jest zamknięta - zestawienie różnych interfejsów programowania aplikacji dostępnych w przeglądarkach internetowych znajdziesz tutaj: MDN - Web APIs.

Ciekawą technologią jest także WebAssembly, która w swych założeniach pozwalać ma na znaczące przyspieszenie programów uruchamianych po stronie klienta dzięki rezygnacji z interpretowanego kodu JavaScript na rzecz wydajnego kodu zapisanego w formacie binarnym. Na sieci znajdziesz wiele artykułów dotyczących WebAssembly, ale odczuwasz zaintrygowanie tematem, możesz zacząć lekturę od artykułu na portalu MDN - WebAssembly Concepts.

To wszystko razem sprawia, że twórcy serwisów udostępnianych przez serwery HTTP coraz częściej odchodzą od konwencji udostępniania treści w formie >>stron<< (bardziej lub mniej statycznych) na rzecz interaktywnych >>aplikacji<< internetowych. Pod pojęciem aplikacji internetowych będziemy więc rozumieli takie usługi udostępniane dla przeglądarek internetowych, których funkcjonowanie opiera się nie tylko na prezentacji treści, ale także zapewniają interaktywność z użytkownikiem. Przykładowo, aplikacją nazwiemy działającą w przeglądarce wyszukiwarkę połączeń komunikacji miejskiej, narzędzie do zarządzania blogiem (tzw. system CMS (ang. Content Management System), czy grę komputerową. Jako aplikację potraktować można także stronę internetową, która np. ma osadzone elementy interaktywne.

2.6. Mobilne aplikacje internetowe

Szczególnym typem aplikacji internetowych są aplikacje dedykowane do uruchamiania na urządzeniach mobilnych. W ich kontekście często używa się terminu "aplikacje mobilne". To wyróżnienie jest o tyle istotne, że zwykle implementowane są one jako tzw. aplikacje natywne, tzn. przygotowane przy użyciu dedykowanych narzędzi i przygotowane do instalacji na urządzeniach mobilnych. W ramach niniejszego kursu tematyka aplikacji mobilnych natywnych nie będzie w ogóle poruszana. Interesować nas jednak będą aplikacje internetowe mobilne rozumiane jako serwisy uruchamiane w przeglądarkach internetowych na urządzeniach mobilnych, co jest dzisiaj niezwykle powszechne. Tematyka ta zostanie zaadresowana w kontekście projektowania interfejsu użytkownika zgodnie z koncepcją zachowania tzw. responsywności (ang. *Responsive Web Design*).

2.7. Zaawansowane aplikacje internetowe

Cóż więc sprawia, że aplikację internetową możemy nazwać „zaawansowaną”? Zakres tego określenia zdecydowanie ulega zmianie na przestrzeni czasu, jednakże zdaniem Autora aplikacja taka powinna cechować się zaadresowaniem przynajmniej niżej wymienionych kwestii:

- poprawnym użyciem technologii stanowiących budulec stron internetowych w postaci języka HTML5, kaskadowych arkuszy styli CSS3 oraz języka JavaScript,
- wykorzystaniem dobrze wyspecyfikowanego i zaimplementowanego protokołu komunikacji pomiędzy tzw. frontendem a backendem,
- interfejsem użytkownika zaprojektowanym w taki sposób, aby działał poprawnie na przeglądarkach uruchamianych zarówno na urządzeniach stacjonarnych jak i mobilnych (czyli zaprojektowanym zgodnie z koncepcją *Responsive Web Design*),
- odpowiednio zaimplementowanym backendem (w tym dobrze zaprojektowaną bazą danych), w sposób minimalizujący wpływ tzw. wąskich gardeł na szybkość działania aplikacji,
- zapewnieniem bezpieczeństwa i integralności przetwarzanych informacji,
- zapewnienie właściwego balansu pomiędzy przetwarzaniem danych i renderowaniem po stronie backendu jak i frontendu, uwzględniając zarówno kwestie wydajnościowe jak i optymalizację pod kątem wyszukiwarek (ang. *Search Engine Optimization*, SEO)
- zapewnienie dostępności cyfrowej aplikacji,
- i opcjonalnie, uwzględnienie specyfiki uruchamiania aplikacji na urządzeniu mobilnym, np. poprzez implementację jej z wykorzystaniem koncepcji progresywności (ang. *Progressive Web Application*, PWA).

Powyższa lista oczywiście nie jest kompletna. Dużo zależy także od konkretnego zastosowania danej aplikacji - inne oczekiwania będziemy mieli w stosunku do np. serwisu mikroblogowego, aplikacji do planowania podróży czy też aplikacji do współtworzenia treści w czasie rzeczywistym przez wielu użytkowników.

W ramach niniejszego kursu poruszone zostaną wybrane zagadnienia, które pozwalają przybliżyć aplikację internetową do miana "zaawansowanej".

3. Podstawowy podział na warstwy: frontend i backend

Z aplikacji internetowych ludzie korzystają zazwyczaj przy użyciu **przeglądarek internetowych** uruchamianych na ich urządzeniach takich jak komputery osobiste, tablety czy smartfony. Interakcja z aplikacją rozpoczyna się od wpisania w przeglądarkę **adresu**, który użytkownik chce odwiedzić, a następnie rozpoczyna się komunikacja z **serwerem** - przeglądarka wysyła do niego żądania pobrania niezbędnych danych, w szczególności dokumentów HTML, CSS czy kodu JavaScript. W dalszej kolejności pobierane są elementy takie jak czcionki, ilustracje, multimedia.

Taka organizacja wynika wprost z faktu, że najpopularniejsze aplikacje internetowe implementowane są w **architekturze klient-serwer**, z wykorzystaniem protokołu HTTP (ang. *Hypertext Transfer Protocol*) lub HTTPS (ang. *Hypertext Transfer Protocol Secure*). Mamy tu do czynienia z wyraźnym podziałem kodu aplikacji na dwie warstwy: **frontend** i **backend**, a także z podziałem architektonicznym wynikającym z miejsca uruchamiania tego kodu: klient i serwer.

Frontend obejmuje warstwę prezentacyjną, czyli wszystko, co jest odpowiedzialne za wygląd i interakcję aplikacji z użytkownikiem. Z kolei backend to "niewidoczna" część aplikacji, która jest odpowiedzialna za logikę biznesową, przechowywanie i przetwarzanie danych a także za komunikację (integrację) z systemami zewnętrznymi. Należy zwrócić uwagę Czytelnika, że obecnie nie można już jednoznacznie utożsamiać zagadnień związanych z wizualną stroną aplikacji wyłącznie ze stroną kliencką - otóż w ostatnich latach granica backend/frontend wyraźnie zaciera się. W szczególności wyróżnia się w nomenklaturze techniki renderowania widoku (czyli generowania dokumentu lub elementów dokumentu HTML) po stronie serwera lub klienta.

3.1. Frontend

Frontend to ta część aplikacji, którą użytkownik widzi i z którą bezpośrednio wchodzi w interakcję, tj. układ treści, kolorystyka, typografia, przyciski, formularze, itp. Innymi słowy, frontend obejmuje warstwę prezentacyjną, czyli wszystko, co jest odpowiedzialne za wygląd i interakcję aplikacji z użytkownikiem.

Do podstawowych technologii wykorzystywanych do implementacji frontendu aplikacji należą:

- **HTML (ang. *HyperText Markup Language*):** służy do opisu struktury strony internetowej; opis składa się z elementów takich jak np. nagłówki, akapity, ilustracje.
- **CSS (ang. *Cascading Style Sheets*):** odpowiada za wygląd elementów opisanych w dokumencie HTML, czyli cechy takie jak rozmieszczenie, rozmiar, kolor, czcionka.
- **JavaScript:** pozwala zaimplementować interaktywność strony internetowej w postaci np. dynamicznych zmian treści, animacji, obsługi zdarzeń (np. reakcji na kliknięcie myszką).

Frontend odpowiada za bezpośrednie doświadczenie użytkownika, czyli za to, jak aplikacja wygląda i jak się zachowuje na urządzeniu użytkownika (np. komputerze, smartfonie). Projektant frontendu powinien zwrócić szczególną uwagę na te elementy, z którymi użytkownik wchodzi w interakcję, np. menu rozwijane, przyciski, formularze, treści aktualizowane w dynamiczny sposób, tj. bez przeładowywania całego okna dokumentu HTML. Ich poprawna obsługa, cechująca się intuicyjnością, czytelnością, stabilnością i krótkim czasem reakcji składa się na pojęcie tzw. User Experience (UX), czyli zespołu cech aplikacji odbieranych całościowo przez użytkownika końcowego w sposób subiektywny.

Choć technicznie rzecz biorąc jest możliwe wykorzystanie języka JavaScript do implementacji interaktywności bez wykorzystania żadnego tzw. frameworku czy biblioteki, w praktyce byłoby to dość pracochłonne i wtórne. Od kilkunastu lat w powszechnym użyciu są narzędzia ułatwiające budowanie interfejsów użytkownika, np. **React**, **Angular**, **Vue.js**. Wrócimy do nich w dalszej części podręcznika.

3.2. Backend

Backend to "niewidoczna" część aplikacji, która jest odpowiedzialna za logikę biznesową, przechowywanie i przetwarzanie danych a także za komunikację (integrację) z systemami zewnętrznymi.

Kluczowe aspekty backendu:

- **Serwery**: Backend działa na serwerach, które obsługują żądania przesyłane przez przeglądarki internetowe (np. pobranie danych do wyświetlenia na stronie).
- **Bazy danych**: Backend obsługuje komunikację z bazą danych, zapisując, pobierając i przetwarzając dane. Popularne bazy danych to np. **MySQL**, **PostgreSQL**, **MongoDB**.
- **Logika biznesowa**: Backend implementuje logikę działania aplikacji, np. uwierzytelnianie użytkowników, przetwarzanie zamówień w sklepie internetowym, obliczenia matematyczne, generowanie raportów itp.
- **API (Application Programming Interface)**: Backend dostarcza interfejsy API, które pozwalają frontendowi na wymianę danych z serwerem. Dobrze zdefiniowane API jednoznacznie określa sposób wykorzystania protokołu **HTTP** i formatów danych jak **JSON** czy **XML** do wymiany informacji pomiędzy frontendem a backendem.
- **Bezpieczeństwo**: Backend odpowiada za autentykację użytkowników aplikacji, autoryzację dostępu do zasobów oraz ochronę danych przed nieautoryzowanym dostępem.
- **Technologie backendowe**:
    - **Języki programowania**: Typowe języki to **Python**, **Java**, **Ruby**, **PHP**, **C#**. Od kilku lat także **JavaScript** przy okazji użycia środowiska Node.js.
    - **Frameworki backendowe**: Ułatwiają budowanie aplikacji serwerowych, np. **Django** (Python), **Spring** (Java), **Express.js** (Node.js), **Ruby on Rails** (Ruby), **Laravel** (PHP).
    - **Język zapytań do baz danych: SQL** (ang. Structured Query Language) służy do definiowania operacji wykonywanych na bazie danych, np. utworzenia, usunięcia, modyfikacji lub dodania rekordu.

Backend jest odpowiedzialny za przetwarzanie danych, zarządzanie nimi oraz zapewnienie, że aplikacja działa zgodnie z określonymi zasadami i logiką.

3.3. Nieostra granica backend-frontend

Historycznie podział architektoniczny aplikacji internetowych na frontend a backend był bardzo czytelny i jednoznaczny, gdyż za rozdzielnym zakresem odpowiedzialności szły także zupełnie odmienne technologie i języki wykorzystywane w obydwu tych obszarach. W ostatnich latach obserwuje się zacieranie tej do niedawna bardzo wyraźnej granicy. Wynika to z kilku przyczyn.

1. Rozwój frameworków JavaScript po stronie klienta i serwera

Frameworki JavaScript, takie jak **Node.js** oraz frameworki frontendowe typu **React**, **Vue.js** czy **Angular**, odegrały kluczową rolę w zacieraniu się granicy między frontendem a backendem. **Node.js** umożliwił programowanie po stronie serwera w JavaScript, czyli języku, który wcześniej był zarezerwowany niemal wyłącznie dla warstwy frontendowej. W ten sposób pojawiła się możliwość używania jednego języka w całym stosie technologicznym, co sprawiło, że granice między backendem a frontendem stały się bardziej płynne.

2. Wzrost popularności aplikacji typu SPA (Single-Page Application)

Aplikacje typu **SPA** (Single-Page Application), takie jak te budowane w **React**, **Vue** lub **Angular**, zacierają granicę między frontendem a backendem, ponieważ frontend przejmuje coraz więcej zadań związanych z logiką aplikacji. Odpowiada za pobieranie danych, ich przetwarzanie i dynamiczne wyświetlanie zawartości bez konieczności przeładowywania całej strony. W takich aplikacjach frontend bardzo często korzysta z API (np. REST lub GraphQL), a backend staje się bardziej wyspecjalizowanym "serwerem danych", dostarczającym tylko surowe dane w odpowiedzi na żądania klienta.

3. Popularność architektury mikroserwisowej i API-first

Wzrost popularności **mikroserwisów** i podejścia **API-first** przyczynia się do zmniejszenia wyraźnej granicy między frontendem a backendem. W architekturze mikroserwisowej:

- Backend jest podzielony na mniejsze, niezależne serwisy, które komunikują się za pośrednictwem API. Każdy mikroserwis jest odpowiedzialny za konkretną funkcjonalność i często może być rozwijany w różnych językach programowania.
- Frontend, dzięki dostępowi do API, może w bardziej elastyczny sposób korzystać z danych i funkcji backendu. W takim modelu backend dostarcza tylko dane w formie API, a cała logika wyświetlania, obsługi zdarzeń i przetwarzania danych dzieje się w przeglądarce.

Przykładem są architektury typu **JAMstack**, gdzie frontend może być hostowany osobno, a backend jest serią rozproszonych usług, które dostarczają dane za pośrednictwem API.

https://www.cloudflare.com/learning/performance/what-is-jamstack/

4. Wzrost popularności tzw. "Full-stack developerów"

Jeszcze kilka lat temu specjalizacje frontend i backend były wyraźnie rozdzielone, a osoby pracujące nad jednym z tych obszarów rzadko przekraczały granicę drugiego. Obecnie jednak wzrasta popularność roli **full-stack developera**, który zna zarówno technologie backendowe, jak i frontendowe. Full-stack developerzy są w stanie pracować nad wszystkimi warstwami aplikacji, co naturalnie zmniejsza dystans między obiema stronami.

Dzięki narzędziom takim jak **Node.js**, **Express.js** (backend) oraz **React** czy **Vue** (frontend), full-stack developerzy mogą tworzyć kompletne aplikacje od początku do końca, mając pełną kontrolę nad przepływem danych i interakcjami między frontendem a backendem.

5. Pojawienie się narzędzi do renderowania po stronie serwera (SSR) w aplikacjach frontendowych

Nowoczesne frameworki frontendowe, takie jak **Next.js** (dla Reacta) i **Nuxt.js** (dla Vue.js), wprowadziły łatwo dostępne mechanizmy renderowania po stronie serwera (ang. SSR - Server-Side Rendering). Umożliwia to tworzenie aplikacji, które są renderowane zarówno na serwerze (poprawiając SEO i szybkość ładowania), jak i w przeglądarce (zapewniając interaktywność i elastyczność). Dzięki temu podejściu jedna aplikacja może obsługiwać zarówno logikę serwerową, jak i klientową, co dodatkowo zmniejsza podział na backend i frontend.

3.4. SSR – Server Side Rendering

Renderowanie po stronie serwera polega na tym, że cały HTML, CSS i (częściowo) JavaScript są generowane na serwerze, zanim strona trafi do przeglądarki użytkownika. Gdy użytkownik otwiera stronę, serwer przygotowuje już w pełni wyrenderowany kod HTML, który następnie jest przesyłany do przeglądarki.

Jak to działa:

1. Przeglądarka internetowa wysyła zapytanie do serwera, np. wpisując adres URL strony.
2. Serwer przetwarza żądanie, wykonuje niezbędne operacje (np. pobiera dane z bazy) i generuje gotowy HTML.
3. Serwer wysyła ten HTML do przeglądarki.
4. Użytkownik od razu widzi stronę (nawet jeśli niektóre interaktywne elementy mogą być ładowane później poprzez JavaScript).

Przykłady technologii wykorzystujących SSR:

- **PHP**, **Ruby on Rails**, **Django** (frameworki backendowe z renderowaniem HTML na serwerze),
- **Next.js** (framework do SSR dla Reacta),
- **Nuxt.js** (SSR dla Vue.js).

Zalety SSR:

- **Szybsze renderowanie początkowe**: HTML jest gotowy już na etapie serwera, więc użytkownik widzi treść strony szybciej.
- **SEO**: Ponieważ wyszukiwarki, takie jak Google, mogą łatwiej indeksować strony z pełnym HTML-em, SSR jest korzystniejszy pod kątem optymalizacji dla wyszukiwarek (SEO).
- **Lepsza wydajność dla słabszych urządzeń**: Rendering odbywa się na serwerze, więc nawet użytkownicy na wolniejszych urządzeniach mogą szybko zobaczyć stronę.

Wady SSR:

- **Obciążenie serwera**: Serwer musi renderować HTML dla każdego zapytania, co może być kosztowne pod względem zasobów, zwłaszcza przy dużym ruchu.
- **Dłuższe czasy odpowiedzi na dynamiczne treści**: Serwer musi każdorazowo generować nową stronę, co może wydłużać czas odpowiedzi przy bardzo dynamicznych aplikacjach.

3.5. CSR – Client Side Rendering

Renderowanie po stronie klienta (przeglądarki) w podstawowym ujęciu polega na tym, że serwer wysyła do klienta minimalny szablon HTML oraz kod JavaScript i CSS. JavaScript odpowiada za dynamiczne generowanie treści po stronie przeglądarki internetowej. Po załadowaniu strony przeglądarka w osobnych żądaniach HTTP pobiera dane (np. przez API) i na ich podstawie renderuje widok. Istotną różnicą w stosunku do SSR jest „składanie" elementów HTML w docelowy dokument w przeglądarce, a nie na serwerze.

Popularne frameworki **React**, **Vue.js**, **Angular** pracują w trybie CSR i pozwalają budować aplikacje w stylu SPA (ang. Single-Page Application), czy z dynamicznym ładowaniem zawartości po stronie klienta.

Zalety CSR:

- **Lepsza interaktywność**: Aplikacje renderowane po stronie klienta mogą być bardzo dynamiczne i interaktywne (typowe dla aplikacji jednostronowych, czyli SPA), z płynnymi przejściami między widokami.
- **Mniejsze obciążenie serwera**: Serwer nie musi generować pełnych stron HTML, a jedynie odpowiadać na żądania API, co zmniejsza obciążenie.
- **Płynniejsze wrażenia dla użytkownika**: Po załadowaniu strony dalsze przejścia między stronami (widokami) są szybsze, ponieważ wszystko jest już załadowane i przetwarzane lokalnie w przeglądarce.

Wady CSR:

- **Wolniejsze renderowanie początkowe**: Użytkownik może zobaczyć "pustą" stronę (szablon) ekran ładowania aplikacji zanim JavaScript pobierze i wyrenderuje treść.
- **SEO**: Ponieważ HTML generowany jest dynamicznie w przeglądarce, tradycyjne roboty wyszukiwarek mogą mieć problem z indeksowaniem treści, chyba że zostanie użyty specjalny mechanizm (np. prerendering).
- **Obciążenie przeglądarki**: Więcej zasobów przetwarzania przeniesione jest na przeglądarkę, co może powodować wolniejsze działanie na starszych lub słabszych urządzeniach.

3.6. Klient

W przypadku aplikacji internetowych rolę **klienta** najczęściej pełni **przeglądarka internetowa**.

Uwaga

W mowie potocznej bardzo powszechne jest mylenie pojęć **przeglądarka** i **wyszukiwarka** internetowa.

Przeglądarka internetowa (ang. *web browser*):

- aplikacja uruchamiana na komputerze / tablecie / telefonie, która umożliwia użytkownikowi wyświetlanie i interakcję ze stronami internetowymi WWW,
- przykłady:
  - Google Chrome,
  - Mozilla Firefox,
  - Microsoft Edge,
  - Safari,
  - Opera

Wyszukiwarka internetowa (ang. *search engine*):

- usługa dostępna online, która umożliwia wyszukiwanie informacji w Internecie; dostarcza użytkownikowi listę wyników (odnośników do stron internetowych) na podstawie wprowadzonego zapytania (słów kluczowych); wyniki generowane są na podstawie indeksu stron internetowych,
- przykłady:
  - Google,
  - Bing,
  - Yahoo!,
  - DuckDuckGo,
  - Baidu

Z punktu widzenia twórcy aplikacji internetowych, kluczowym elementem przeglądarki odpowiedzialnym za interpretację, renderowanie i wyświetlanie stron internetowych jest silnik przetwarzania HTML, tzw. silnik renderujący (ang. *render engine*). Silnik ten przekształca kod HTML, CSS, JavaScript i inne zasoby w interaktywny interfejs aplikacji prezentowany użytkownikowi. Obecnie (rok 2024) większość popularnych przeglądarek korzysta z projektu Chromium rozwijanego przez Google. W ramach Chromium głównym silnikiem renderującym jest **Blink**, który współpracuje z **V8** (silnik JavaScript) i innymi komponentami przeglądarki. Testując swoją aplikację, warto sprawdzić jej zachowanie i wygląd w przeglądarkach wyposażonych w alternatywne silniki, na przykład:

- **Mozilla Firefox**
  - silnik renderujący: **Gecko**
  - silnik JavaScript: **SpiderMonkey**
  - inne przeglądarki korzystające z silnika Gecko
- **Apple Safari**
  - silnik renderujący: **WebKit**
  - silnik JavaScript: **JavaScriptCore / Nitro**
- **Konqueror**
  - silnik renderujący: **WebKit**, w przeszłości KHTML
- **GNOME Web (Epiphany)**, **Eolie**, **Tangram**
  - silnik renderujący: **WebKitGTK**

4. Architektury aplikacji internetowych

Wzorce architektoniczne to schematy organizacyjne, które definiują sposób, w jaki komponenty aplikacji współpracują ze sobą na poziomie systemu. W kontekście aplikacji internetowych, wybór odpowiedniego wzorca architektonicznego ma kluczowe znaczenie dla wydajności, skalowalności, łatwości utrzymania i rozwoju aplikacji. W niniejszym rozdziale omówiono skrótowo najważniejsze wzorce architektoniczne stosowane przy tworzeniu aplikacji internetowych:

- **architektura klient - serwer**,
- **architektura monolityczna**,
- **architektura oparta o mikroserwisy**.

4.1. Architektura klient - serwer

Architektura klient-serwer jest jednym z najbardziej powszechnych modeli komunikacyjnych w informatyce, szczególnie w kontekście aplikacji internetowych. Opiera się na interakcji między dwoma głównymi elementami: **klientem** i **serwerem**. Powszechnie wykorzystywany w środowisku aplikacji webowych protokół HTTP (ang. *Hypertext Transfer Protocol*) został zaprojektowany do pracy właśnie w architekturze klient - serwer. Tym samym z architekturą klient - serwer mamy do czynienia praktycznie w każdej aplikacji internetowej.

Zasadnicze role w architekturze klient - serwer

Klient

Klient to aplikacja lub urządzenie, które inicjuje połączenie i wysyła żądania do serwera. W kontekście protokołu HTTP, klientem zazwyczaj jest przeglądarka internetowa (np. Google Chrome, Mozilla Firefox), ale może też nim być aplikacja mobilna, program komputerowy, czy nawet urządzenie klasy IoT (Internet of Things). Co do zasady klient nie przechowuje danych ani nie implementuje logiki biznesowej (z drobnymi wyjątkami). Rolą klienta jest zazwyczaj interakcja z użytkownikiem, przesyłanie żądań do serwera oraz prezentacja danych w interfejsie użytkownika.

Serwer

Serwer ma postać systemu komputerowego z uruchomionym odpowiednim oprogramowaniem, dołączonego do sieci internetowej lub lokalnej, i oczekującego na połączenia inicjowane przez klientów. Serwer odbiera żądania od klientów, przetwarza je i odsyła odpowiedzi. Serwer przechowuje dane, pliki, i inne zasoby, a także implementuje logikę aplikacji.

Przeczytaj także: **rozdział poświęcony tworzeniu oprogramowania dla serwera (tzw. backend)**

4.2. Architektura monolityczna

Architektura monolityczna (potocznie: monolit) to jeden z tradycyjnych wzorców projektowania aplikacji, w którym wszystkie komponenty aplikacji (front-end, back-end, logika biznesowa, bazy danych) są zintegrowane w jeden zwarty program komputerowy. Oznacza to, że cała aplikacja jest zbudowana i wdrażana jako jedna całość.

Podejście takie ma szereg **zalet**:

1. Prostota rozwoju i wdrożenia: w początkowej fazie rozwoju projektu, architektura monolityczna jest łatwa do zrozumienia i wdrożenia, ponieważ wszystkie komponenty są w jednym miejscu. Nie ma potrzeby zarządzania komunikacją między usługami, jak ma to miejsce w bardziej skomplikowanych architekturach.
2. W przypadku prostych projektów, aplikacje monolityczne są łatwiejsze do testowania, ponieważ kod odpowiedzialny za wszystkie funkcjonalności znajduje się w jednym miejscu. Testy end-to-end mogą być wykonywane w jednym środowisku bez potrzeby symulowania komunikacji między usługami.
3. Jedna aplikacja oznacza jeden punkt wejścia i łatwiejsze zarządzanie stanem aplikacji, co zmniejsza ryzyko wystąpienia błędów komunikacji między modułami.
4. Na wczesnym etapie, gdy aplikacja jest jeszcze niewielka, rozwój może być szybki, ponieważ nie ma potrzeby dzielenia aplikacji na mniejsze serwisy.

Jednakże wraz z rozwojem aplikacji, ujawniają się **wady** architektury monolitycznej:

1. Skalowanie w architekturze monolitycznej polega zazwyczaj na powielaniu całej aplikacji na nowe serwery. Można skalować tylko całą aplikację, co jest mało efektywne, gdy wyłącznie jeden z modułów staje się wąskim gardłem.
2. Ścisłe powiązania pomiędzy komponentami utrudniają rozwój i wprowadzanie zmian w aplikacji, gdyż zmiany w jednym module mogą wymagać zmian w innych częściach aplikacji. Praca z całym kodem aplikacji ponadto zwiększa ryzyko wprowadzenia błędów.
3. Utrudnione są wdrożenia i aktualizacje, gdyż każda zmiana w kodzie powinna pociągać za sobą przetestowanie całej aplikacji, co jest czasochłonne i ryzykowne.
4. Wraz z rozwojem aplikacji, kod staje się coraz bardziej złożony, co utrudnia jego zrozumienie, utrzymanie i wprowadzanie zmian przez nowych programistów.
5. Błąd w jednym module aplikacji może wpłynąć na całą aplikację, ponieważ wszystkie komponenty działają razem jako jeden proces.

Architektura monolityczna jest prostym i łatwym do wdrożenia rozwiązaniem na początkowym etapie tworzenia aplikacji. Jednak w miarę jak aplikacja rośnie, staje się ona coraz bardziej złożona, trudna do zarządzania, skalowania i utrzymania. Dlatego w bardziej zaawansowanych projektach często rozważa się przejście na architektury bardziej modularne, takie jak mikroserwisy, które rozwiązują wiele problemów związanych z monolitem, oferując lepszą skalowalność i elastyczność.

4.3. Mikroserwisy

Architektura mikroserwisowa polega na podziale aplikacji na małe, autonomiczne jednostki (mikroserwisy), które są wdrażane, skalowane i zarządzane niezależnie od siebie. Każdy mikroserwis ma własną logikę biznesową, bazę danych (lub dostęp do swojej części danych) i jest zaprojektowany tak, aby działać jako niezależna jednostka.

Do **zalet** architektury mikroserwisowej zaliczyć należy:

1. **Łatwość skalowania.** Mikroserwisy można skalować niezależnie od siebie, co pozwala na lepsze wykorzystanie zasobów i elastyczne reagowanie na zmieniające się obciążenie poszczególnych komponentów aplikacji.
2. **Szybsze wdrożenie i rozwój.** Dzięki niezależności serwisów zespoły programistów mogą równocześnie rozwijać różne części aplikacji, co przyspiesza wdrażanie nowych funkcji.
3. **Elastyczność w doborze technologii.** Możliwość użycia różnych technologii i języków programowania w poszczególnych mikroserwisach pozwala na ich optymalny wybór w zależności od potrzeb danego mikroserwisu.
4. **Odporność na awarie**. Awaria jednego mikroserwisu nie musi zablokować działania całej aplikacji, co zwiększa niezawodność i dostępność systemu.
5. **Łatwiejsze utrzymanie i zarządzanie**. Poszczególne mikroserwisy są znacznie mniejsze niż odpowiadająca im funkcjonalnie aplikacja monolityczna, a więc są też i prostsze w utrzymaniu. Ułatwia to zarządzanie kodem i pozwala na szybsze naprawianie błędów.

Nie ma jednak róży bez kolców. Architektura mikroserwisowa niesie też ze sobą pewne **wady**:

1. **Większa złożoność komunikacji.** Wzrost liczby mikroserwisów prowadzi do skomplikowania komunikacji między nimi, co wymaga zaprojektowania odpowiednich interfejsów API, skonfigurowania połączeń sieciowych oraz rozwiązywania problemów związanych z niezawodnością i opóźnieniami. Najczęściej używa się protokołów HTTP/REST, gRPC lub komunikacji poprzez kolejki komunikatów (np. RabbitMQ, Apache Kafka).
2. **Trudności w zarządzaniu danymi.** Rozproszone bazy danych mogą prowadzić do problemów z transakcyjnością i spójnością danych, co wymaga zastosowania dodatkowych strategii, takich jak zarządzanie zdarzeniami.
3. **Dodatkowe koszty utrzymania.** Monitorowanie, logowanie, śledzenie i zarządzanie wieloma mikroserwisami jest bardziej kosztowne i złożone w porównaniu do architektury monolitycznej.
4. **Wymagane dodatkowe kompetencje.** Mikroserwisy wymagają zaawansowanych umiejętności z rodziny DevOps, znajomości narzędzi do orkiestracji (np. Kubernetes) oraz odpowiednich strategii monitorowania i testowania aplikacji webowych.

5. Wzorce projektowe

Wzorce projektowe to sprawdzone rozwiązania typowych problemów projektowych, które pojawiają się podczas tworzenia oprogramowania. Są to powszechnie uznane metody projektowania aplikacji, ich części składowych, interfejsów oraz interakcji pomiędzy nimi. Posługiwanie się wzorcami pomaga programistom tworzyć oprogramowanie, którego kod źródłowy jest czytelny i łatwe w utrzymaniu. Należy pamiętać, że wzorce projektowe nie są gotowymi fragmentami kodu do bezpośredniego użycia, lecz raczej wytycznymi czy "szablonami", które można dostosować do specyficznych potrzeb danego projektu.

Korzyści ze stosowania wzorców projektowych

1. **Rozwiązywanie powtarzalnych problemów**: Wzorce projektowe oferują rozwiązania problemów, które pojawiają się w wielu projektach. Zamiast wymyślać rozwiązanie od podstaw, można zastosować wzorzec, który oferuje gotowe rozwiązanie.

2. **Poprawa jakości kodu**: Dzięki wzorcom projektowym kod jest bardziej zrozumiały, lepiej zorganizowany i łatwiejszy w utrzymaniu. Wzorce promują dobre praktyki, takie jak separacja zadań czy zasadę pojedynczej odpowiedzialności.

3. **Ułatwienie komunikacji w zespole**: Wzorce projektowe umożliwiają programistom wzajemną komunikację za pomocą wspólnego języka. Mówiąc o wzorcu, takim jak „Singleton” czy „Obserwator”, członkowie zespołu od razu wiedzą, o jakiej strukturze kodu i funkcjonalności mowa, co przyspiesza proces projektowania i rozwoju aplikacji.

4. **Reużywalność i elastyczność kodu**: Kod oparty na wzorcach jest często bardziej modularny i łatwiejszy do ponownego użycia w innych częściach aplikacji lub w innych projektach. Dzięki dobrze zastosowanym wzorcom projektowym, zmiany w jednej części systemu nie wpływają negatywnie na inne jego części.

5. **Ułatwione testowanie**: Wzorce projektowe, takie jak „Factory” czy „Dependency Injection”, ułatwiają testowanie aplikacji przez umożliwienie łatwej zamiany zależności na atrapy/makiety (mocks) lub inne implementacje w testach jednostkowych.

6. **Redukcja złożoności**: Dzięki wzorcom możliwe jest uporządkowanie i uproszczenie złożonych zależności w aplikacji. Wzorce takie jak „Fasada” czy „Adapter” pozwalają na ukrycie skomplikowanych obiektów i interakcji za prostymi interfejsami.

Temat wzorców projektowych jest niezmiernie obszerny i zdecydowanie wykracza poza ramy niniejszego podręcznika. Zainteresowanego Czytelnika warto odesłać do strony Refactoring Guru, w ramach której najpopularniejsze wzorce projektowe mające zastosowanie tworzeniu różnego rodzaju oprogramowania (nie tylko stricte aplikacji internetowych).

W następujących podrozdziałach omówiono pokrótce wybrane wzorce, których świadomość ułatwia twórcy aplikacji internetowych podejmowanie odpowiednich decyzji projektowych.

5.1. BBOM - Big Ball of Mud

Big Ball of Mud (BBOM) to ironiczna nazwa dla antywzorca projektowego, który opisuje oprogramowanie pozbawione wyraźnej struktury, czytelnej architektury i dobrych praktyk programistycznych. Jest to termin używany do opisania kodu, który w miarę rozwoju projektu staje się coraz bardziej złożony, trudny do zrozumienia, utrzymania i rozwijania. Tak niestety często wyglądają projekty początkujących programistów lub zespołów, w których nie dba się o długoterminowe utrzymanie wysokiej jakości kodu. Jakie cechy charakteryzują takie projekty i jakie to niesie konsekwencje?

- Brak spójnej architektury. Kod nie ma czytelnie określonej struktury ani podziału na warstwy. Granice pomiędzy komponentami są zatarte, co w skutkuje licznymi wzajemnymi zależnościami i sprzężeniami. Znacząco utrudnia to m.in. selektywne testowanie poszczególnych bloków kodu.
- Brak separacji odpowiedzialności. Poszczególne funkcje są przerośnięte, a ich odpowiedzialność obejmuje więcej niż jedno ściśle zdefiniowane zadanie. W rezultacie zmiana jednej części aplikacji może mieć nieprzewidywalny wpływ na inną jej część, a w dalszej konsekwencji utrzymywanie takie systemu staje się kosztowne, gdyż wymaga dodatkowego wysiłku. W skrajnym przypadku zespół programistów stopniowo traci kontrolę nad złożonością systemu, a nowi członkowie zespołu mają trudności z orientacją w kodzie źródłowym.
- W toku rozwoju aplikacji stosowane bywają różnego rodzaju łatki i obejścia o tymczasowym charakterze. W dłuższej perspektywie takie nieprzemyślane rozwiązania jedynie nawarstwiają problemy i prowadzą do jeszcze większej złożoności.
- Skomplikowana struktura kodu skutkuje brakiem możliwości przygotowania sensownych testów jednostkowych. W rezultacie kod jest często wcale lub słabo przetestowany. Tym samym rośnie ryzyko błędów, awarii i problemów z użytkowaniem.

Aby uniknąć wpadnięcia i ugrzęźnięcia w "wielkiej kuli błota", należy zadbać o:

- zainwestowanie czasu w zaplanowanie architektury systemu oraz podział na odpowiednie warstwy i moduły.
- regularną refaktoryzację,
- stosowanie wzorców architektonicznych takich jak np. MVC, MVVM, czy mikroserwisy,
- testy jednostkowe i integracyjne, które pokrywają logikę aplikacji - pozwala to na bezpieczne wprowadzanie zmian i ogranicza narastanie tzw. długu technicznego,
- praktykowanie kultury tzw. "code review", czyli regularnego przeglądania zwłaszcza nowego kodu przez innych członków zespołu - pomaga to w utrzymaniu jakości, wymianie wiedzy w zespole, a także identyfikacji problemów,
- prowadzenie aktualnej dokumentacji oraz bieżące komentowanie kodu źródłowego, co ułatwia zrozumieć intencje projektowe i logikę - jest to szczególnie cenne dla nowych członków zespołu, a także w sytuacjach, gdy po dłuższym czasie pojawia się potrzeba wprowadzenia zmian w kodzie, nad którym przez dłuższy czas już nikt nie pracował lub jego twórcy np. zmienili miejsce pracy.

Wspomniana wyżej refaktoryzacja sama w sobie obejmuje wiele zagadnień. Zainteresowanego Czytelnika zapraszam do ponownego odwiedzenia strony Refactoring Guru - tym razem do działu poświęconego refaktoryzacji.

5.2. MVC - Model View Controller

Model-View-Controller (MVC) to popularny wzorzec projektowy stosowany w budowie aplikacji internetowych, który ma na celu oddzielenie logiki biznesowej od logiki prezentacji i zarządzania interakcjami użytkownika. Wzorzec MVC dzieli kod źródłowy aplikacji na trzy zasadnicze warstwy: model (M - Model), widok (V - View), kontroler (C - Controller).

Model

Reprezentuje dane oraz logikę biznesową aplikacji. W ramach modelu implementowane są wszystkie operacje związane z danymi, takie jak ich tworzenie, odczytywanie, aktualizacja i usuwanie (tzw. operacje CRUD - Create, Read, Update, Delete), a także ich walidacja. Tym samym w ramach modelu implementuje się dostęp do bazy danych. Model powinien być niezależny od interfejsu użytkownika, tj. operować na danych w postaci kanonicznej (np. liczby, struktury, tablice), ale nie w postaci sformatowanej dla konkretnego sposobu wyświetlania (np. w postaci tagów języka HTML).

Widok

Jest odpowiedzialny za prezentację danych, czyli za to, co użytkownik widzi na ekranie. Widok generuje interfejs użytkownika i wyświetla dane dostarczane przez Model w sposób zrozumiały dla użytkownika, np. poprzez wygenerowanie odpowiednio sformatowanej strony w języku HTML bądź jej części w postaci poszczególnych komponentów (np. listy zalogowanych użytkowników). Jego rola nie obejmuje natomiast zarządzania logiką aplikacji ani operacjami na danych.

Kontroler

Funkcjonuje jako pośrednik między Modelem a Widokiem. Kontroler przyjmuje żądania przychodzące od przeglądarki internetowej, przekazuje je do odpowiedniego Modelu, a następnie wybiera odpowiedni Widok, aby zwrócić do przeglądarki odpowiednio sformatowane wyniki. Kontroler organizuje przepływ danych i sterowania między Modelem a Widokiem.

Przykład

1. Klient (przeglądarka internetowa) wysyła do serwera żądanie (np. w wyniku kliknięcia w link lub przesłanie formularza) do serwera.
2. Kontroler odbiera żądanie, interpretuje je i decyduje, jakie operacje muszą być wykonane, aby odpowiedzieć na żądanie użytkownika. Na przykład, jeśli użytkownik chce zobaczyć listę produktów, Kontroler wywoła na odpowiednim Modelu metodę, która zwróci niezbędne dane.
3. Model pobiera dane z odpowiedniej tabeli z bazy, wykonuje operacje biznesowe (np. obliczenia, przetwarzanie danych, agregację z różnych tabel itp.). Alternatywnie, w przypadku zapisu, dostarczone dane weryfikuje (np. sprawdza, czy wartości mieszczą się w zakresie wynikającym z logiki aplikacji), a następnie zapisuje we właściwe miejsce w bazie. Operacje przetwarzania danych czasami bywają skomplikowane, ale nie mogą obejmować formatowania pod konkretny sposób prezentacji.
4. Po poprawnym wykonaniu operacji, Model zwraca do Kontrolera pobrane dane. W przypadku wystąpienia błędu (np. żądanie nieistniejącego rekordu lub dostarczenie danych niespełniających kryteriów walidacji), model sygnalizuje błąd - np. poprzez wykorzystanie mechanizmu wyjątków (ang. Exceptions).
5. Kontroler wybiera Widok, który pozwala wyświetlić dane (lub komunikat błędu), i przekazuje do niego dane przekazane przez Model.
6. Widok renderuje dane dostarczone przez Kontroler, generując najczęściej kod HTML, który jest wyświetlany użytkownikowi w przeglądarce.

Zalety

- Rozdzielenie odpowiedzialności. Dzięki MVC, kod aplikacji jest podzielony na trzy główne części, które są odpowiedzialne za różne aspekty działania aplikacji – Model za logikę i dane, View za prezentację, a Controller za obsługę interakcji. Dzięki temu każda z tych części może być rozwijana, testowana i utrzymywana niezależnie od siebie - przez różne zespoły programistów.
- Dzięki wyraźnemu rozdzieleniu odpowiedzialności, aplikacja jest łatwiejsza w utrzymaniu - zmiany w jednym komponencie (np. w definicji wyglądu) nie wpływają bezpośrednio na inne części systemu (np. na logikę biznesową zaimplementowaną w Modelu).
- Takie odseparowane komponenty aplikacji mogą być ponownie używane w innych częściach aplikacji lub nawet w innych projektach, co przyspiesza rozwój i zmniejsza wymagany nakład pracy.
- Rozdzielenie logiki biznesowej od logiki prezentacji pozwala na dużo łatwiejsze testowanie Modelu i Kontrolera, np. bez konieczności uruchamiania interfejsu użytkownika.
- MVC pozwala na łatwe skalowanie poprzez dodawanie nowych funkcjonalności bez konieczności analizowania i modyfikowania całego kodu źródłowego aplikacji.

5.3. MVVM - Model View ViewModel

Model View ViewModel to popularny wzorzec projektowy wykorzystywany przy tworzeniu aplikacji internetowych, szczególnie w kontekście aplikacji ze złożonym interfejsem użytkownika. Jest to rozszerzenie wzorca MVC - Model View Controller, stąd duże podobieństwa pomiędzy nimi. Aby uwypuklić najważniejsze cechy charakterystyczne wzorca MVVM, przedstawiono je w formie porównania z cechami wzorca MVC.

Struktura wzorca

Warstwy modelu (M) oraz widoku (V) są zasadniczo takie same w obydwu wzorcach. Różnica jest w ostatnim członie:

- Kontroler (C) w MVC działa jako mediator pomiędzy widokiem a modelem. Przyjmuje żądania od użytkownika, manipuluje modelem i wywołuje wygenerowanie aktualnego widoku.
- Model widoku (VM) w MVVM reprezentuje warstwę pośrednią, która przetwarza dane z modelu i udostępnia je dla widoku. Można powiedzieć, że model widoku jest abstrakcją widoku eksponującą publiczne właściwości i metody. Co więcej, umożliwia on dwustronne wiązanie danych z widokiem, co pozwala na automatyczną synchronizację danych między widokiem a modelem.

Komunikacja między komponentami

Główną cechą MVVM jest dwukierunkowe wiązanie danych (ang. *two-way data binding*) między widokiem a modelem widoku. Oznacza to, że zmiany w danych w modelu widoku automatycznie aktualizują widok, i na odwrót: zmiany w widoku (np. zmiany wartości w polu tekstowym) automatycznie aktualizują model widoku. Dzięki użyciu dwustronnego wiązania danych, MVVM redukuje ilość kodu niezbędnego do synchronizacji modelu i interfejsu użytkownika. Ze względu na to, że model widoku nie zna bezpośrednio struktury widoku, możliwe jest lepsze rozdzielenie logiki od prezentacji.
Dla kontrastu, w przypadku MVC kontroler bezpośrednio wywołuje odpowiednie metody z widoku, co skutkuje silnymi zależnościami pomiędzy tymi warstwami.

Ułatwienie testowania

W przypadku MVC, kontroler i model są łatwiejsze do przetestowania, ponieważ są niezależne od widoku. Jednak z powodu silnego połączenia między kontrolerem i widokiem, testowanie logiki związanej z interakcjami użytkownika może być trudniejsze. Z kolei wzorzec MVVM promuje łatwość pisania testów jednostkowych i integracyjnych dla logiki prezentacji. Dzięki temu, że model widoku nie jest związany bezpośrednio z widokiem, jest znacznie łatwiejszy do testowania niż kontroler w MVC.

Zastosowanie i kontekst użycia

MVC jest szeroko stosowany w aplikacjach internetowych, np. we frameworkach takich jak Ruby on Rails, ASP.NET MVC, czy Spring MVC. Nadaje się do aplikacji, gdzie interakcje z użytkownikiem są mniej skomplikowane i nie wymagają intensywnego wiązania danych.
Z kolei MVVM jest często stosowany w aplikacjach, które wymagają zaawansowanego interfejsu użytkownika, z dużą ilością interakcji, np. aplikacje desktopowe (WPF, UWP) oraz aplikacje mobilne. Często jest stosowany z frameworkami frontendowymi, takimi jak Angular, Vue.js, czy Knockout.js, gdzie mechanizmy wiązania danych są silnie wspierane.

Dwustronne wiązanie danych

Wzorzec MVC nie posiada wbudowanego mechanizmu dwustronnego wiązania danych. Aktualizacje widoku muszą być wywoływane jawnie przez kontroler. W przypadku MVVM dwustronne wiązanie danych jest centralnym elementem MVVM, co zapewnia synchronizację danych pomiędzy modelem a widokiem, a w konsekwencji znacząco upraszcza implementację złożonych interfejsów użytkownika.

Więcej o wzorcu MVVM przeczytasz w dostępnej online książce Michael Stonis, Enterprise Application Patterns using .NET MAUI, Microsoft Developer Division, 2022.

6. Protokoły komunikacyjne

Sprawna i niezawodna wymiana informacji w nowoczesnych, zaawansowanych aplikacjach internetowych, jest kluczem do uzyskania wysokiej wydajności, responsywności i poziomu zadowolenia użytkowników. Z tego względu wykorzystuje się protokoły i narzędzia komunikacyjne dopasowane do potrzeb poszczególnych elementów aplikacji.

Wśród najpopularniejszych protokołów wykorzystywanych w aplikacjach internetowych wskazać należy:

- HTTP / HTTPS jako podstawowy protokół ekosystemu World-Wide-Web,
- WebSocket,
- gRPC.

6.1. HTTP

Protokół HTTP (ang. *Hypertext transfer protocol*) jest standardem (protokołem) komunikacji internetowej między klientami i serwerami. Został on opracowany i jest nadzorowany przez organizację *World Wide Web Consortium* (W3C, www.w3.org). Najnowsza wersja, HTTP/3, została opublikowana w 2022 roku:

Internet Engineering Task Force, Request for Comments 9114 - HTTP/3

(o protokole HTTP przeczytasz także w tym rozdziale tego podręcznika)

Historia protokołu HTTP sięga początku lat 90-tych XX-wieku. Od tamtego znacząco zmienił się sposób tworzenia usług w sieci Web. Podczas gdy początkowo były to powiązane ze sobą za pomocą odnośników statyczne dokumenty zapisane w języku HTML. Stopniowo jednak sieć ewoluowała w kierunku coraz bardziej dynamicznego dostarczania treści. Istotnie wzrosły też wymagania, np. w obszarze szybkości czy też bezpieczeństwa. Skutkowało to ujawnieniem się pewnych problemów, które wynikają z ograniczeń protokołu, zwłaszcza w jego starszych wersjach. Przyjrzyjmy się krótko tym problemom oraz rozwiązaniom, które można zastosować, aby wyeliminować lub zminimalizować ich skutki.

1. Bezstanowość protokołu HTTP

HTTP jest protokołem bezstanowym, co oznacza, że każde żądanie i odpowiedź są niezależne od innych. Bez zastosowania dodatkowych mechanizmów serwer nie jest w stanie powiązać ze sobą logicznie kolejnych interakcji, nawet jeśli pochodzą od tego samego klienta. Innymi słowy, protokół HTTP nie definiuje pojęcia takiego jak sesja komunikacji z danym klientem. W zdecydowanej większości zachodzi więc konieczność stosowania dodatkowych mechanizmów do zarządzania sesjami, takich jak:

- **ciasteczka (ang. *cookies*)** - są to małe pliki tekstowe zapisywane przez przeglądarkę na urządzeniu użytkownika, służące do przechowywania informacji identyfikujących daną sesję;
  O zastosowaniach ciasteczek przeczytasz w artykule "Po co są ciasteczka?" a także na portalu MDN w artykule "Using HTTP cookies"
- **obsługa sesji po stronie serwera** - serwer musi przechowywać informacje na temat każdej otwartej sesji, aby móc dostarczyć danemu użytkownikowi spersonalizowaną stronę (np. o treści adekwatnej do uprawnień przypisanych do danego konta);
- **tokeny autoryzacyjne**, np. JWT (ang. *JSON Web Token*) - służą do przekazywaniu do klienta informacji m.in. o uwierzytelnionej sesji; więcej o tokenach JWT przeczytasz w tym artykule podręcznika.

2. Wydajność i opóźnienia w HTTP/1.0 i HTTP/1.1

Starsze wersje protokołu HTTP charakteryzują się ograniczeniami wydajnościowymi, które szczególnie uwidaczniają się w przypadku stron i aplikacji, które muszą ładować wiele zasobów, takich jak pliki CSS, obrazy, skrypty JavaScript. Ograniczenia te wynikają bezpośrednio z cech protokołu HTTP:

- **pobieranie jednego zasobu na jedno połączenie TCP** (HTTP/1.0) - wykorzystywany w warstwie transportowej protokół TCP cechuje się pewnym narzutem przy zestawianiu połączenia (wynika to m.in. z zastosowanego tam mechanizmu potwierdzeń znanego jako tzw. three-way handshake); w starej wersji protokołu HTTP otwierane było osobne połączenie dla każdego pobieranego zasobu a następnie połączenie było zamykane; rozwiązanie przyszło wraz z protokołem HTTP/1.1, który pozwolił na przesyłanie kolejno wielu żądań w ramach jednego połączenia TCP;
- **blokowanie sekwencji żądań** (ang. *head-of-line blocking*) (HTTP/1.1) - przesyłane przez przeglądarkę żądania muszą być przetwarzane przez serwer kolejno; wydłużony czas przetwarzania jednego żądania, blokuje możliwość obsługi kolejnych żądań, co w konsekwencji wstrzymuje całą komunikację;
- **brak równoczesności** - aby zminimalizować efekt blokowania sekwencji żądań, w protokole HTTP/1.1. przeglądarki otwierają wiele równoległych połączeń TCP do jednego serwera, co z kolei niepotrzebnie zwiększa obciążenie sieci.

Rozwiązania (częściowo wprowadzone w HTTP/2 i HTTP/3):

- protokół HTTP/2 wprowadza multiplexing, który umożliwia przetwarzanie wielu żądań w jednym połączeniu TCP;
- protokół HTTP/3 eliminuje problem *head-of-line blocking* dzięki wykorzystaniu protokołu QUIC, który w warstwie transportowej korzysta z protokołu UDP w miejsce TCP; więcej o korzyściach płynących ze stosowania protokołu HTTP/3 przeczytasz z artykułu "Nadeszła era HTTP/3. Co wnosi nowy standard komunikacji?"

3. Skalowalność i obciążenie serwera

Wskazane powyżej częste otwieranie i zamykanie połączeń TCP dla każdego żądania (HTTP/1.0) znacząco obciążało serwer w aplikacjach z dużą liczbą użytkowników. W tym aspekcie multipleksing wprowadzony w HTTP/2 również okazał się pomocny. Inną techniką, która pozwala zmniejszyć obciążenie serwera jest wykorzystanie dedykowanej sieci serwerów dostarczających statyczne treści (np. muzykę, filmy itp.), czyli ang. Content Delivery Network (CDN).

4. Problemy z bezpieczeństwem

HTTP w wersji podstawowej przesyła dane w sposób nieszyfrowany, co oznacza, że dane mogą być przechwycone przez atakującego. Ma to szczególne znaczenie np. w przypadku danych logowania, danych kart płatniczych i innych informacji osobistych, ale także ciekawym elementem informacji są same adresy odwiedzanych stron.
Użycie protokołu **HTTPS** (*HTTP Secure*), który dodaje do HTTP protokół szyfrowania TLS (ang. *Transport Layer Security*) rozwiązuje część problemów związanych z bezpieczeństwem, ale nie gwarantuje, że aplikacja internetowa (rozumiana jako całość) korzystająca z tego protokołu jest z automatu w 100% bezpieczna. Zagadnienia bezpieczeństwa są na tyle obszerne, że poświęcony im został cały rozdział niniejszego podręcznika - zapraszam do lektury!

5. Umiarkowana efektywność przesyłania danych

Protokół HTTP/1.1 wymaga przesyłania nagłówków w każdym żądaniu i odpowiedzi. Nagłówki te przesyłane są w formie tekstowej i zajmują średnio po kilkaset bajtów na każde żądanie (czasami więcej, gdy dojdą np. liczne ciasteczka). W przypadku, gdy rozmiar użytecznych danych jest znacząco większy od tej liczby, to nie stanowi to istotnego problemu, jednakże często występują sytuacje, gdy narzut ten nie jest zaniedbywalny (np. pobieranie wielu małych grafik, plików JS, CSS itp.). Konsekwencją jest przede wszystkim dłuższa transmisja danych poprzez sieć, co skutkuje np. dłuższym ładowaniem stron.
Jako rozwiązanie protokół HTTP/2 wprowadza kompresję nagłówków (HPACK), co znacznie zmniejsza ilość danych przesyłanych między klientem a serwerem. Więcej na ten temat przeczytasz w tych artykułach:

- HPACK: the silent killer (feature) of HTTP/2
- dokument RFC7541 - HPACK: Header Compression for HTTP/2 (warto zwrócić uwagę na rozdział 7, który porusza także jeden z problemów bezpieczeństwa, który jest rozwiązywany przez kompresję nagłówków HPACK)

6. Brak wsparcia dla aktualizacji w czasie rzeczywistym

Protokół HTTP został zaprojektowany w oparciu o model żądań i odpowiedzi, gdzie klient wysyła zapytanie, a serwer na nie odpowiada. Nie ma więc natywnego wsparcia dla aktualizacji danych w czasie rzeczywistym ani innych form stałego połączenia między klientem a serwerem. W konsekwencji aplikacje wymagające aktualizacji w czasie rzeczywistym (np. czaty, gry online, strony z notowaniami giełdowymi) muszą korzystać z dodatkowych rozwiązań, takich jak:

- technika **Long Polling** - polega na nawiązaniu połączenia HTTP przez klienta, jednakże odpowiedź nie musi zostać odesłana natychmiast; w niektórych implementacjach serwer może także podtrzymywać połączenie i przesyłać kolejne aktualizacje odpowiedzi w późniejszym czasie,
- protokół WebSocket - jest to uzupełnienie do protokołu HTTP pozwalające na komunikację dwukierunkową; więcej przeczytasz na tej stronie podręcznika,
- Server-Sent Events (SSE) - sformalizowany mechanizm przesyłania komunikatów od serwera do klienta, przy czym najpierw klient musi nawiązać połączenie z serwerem, które następnie jest podtrzymywane; więcej przeczytasz w artykule "Stream updates with server-sent events".

7. Problemy z cache'owaniem

Aby skrócić czas ładowania stron www, protokól HTTP/1.1 wprowadził mechanizmy sterowania procesem cache'owania (np. poprzez nagłówki Cache-Control), czyli przechowywania przez przeglądarkę internetową pobranych już elementów strony. Dzięki temu przy kolejnych odsłonach nie jest potrzebne pobieranie od początku wszystkich zasobów, gdyż przeglądarka wykorzystuje już te zapisane wcześniej w pamięci podręcznej. Wykorzystanie tego mechanizmu daje istotne korzyści, ale wymaga od programisty zaplanowania, m.in. które z zasobów mogą podlegać przechowywaniu przez przeglądarkę, na jak długo oraz co zrobić w przypadku, gdy zasób na serwerze trzeba zaktualizować, ale przeglądarka dostała informację, że może przez pewien czasu nie pobierać danego zasobu z serwera... Wiele szczegółów na ten temat znajdziesz w artykule "HTTP caching" na portalu MDN.

Niniejszy artykuł na pewno nie omawia wszystkich zagadnień związanych bezpośrednio ze specyfiką protokołu HTTP, ale stanowi punkt zaczepienia do dalszych samodzielnych poszukiwań.

6.2. WebSocket

WebSocket to protokół komunikacyjny, który umożliwia dwukierunkową i ciągłą komunikację pomiędzy klientem (np. przeglądarką) a serwerem za pomocą pojedynczego, utrzymywanego w czasie rzeczywistym połączenia. WebSocket został zaprojektowany jako uzupełnienie protokołu HTTP i działa na porcie 80 (HTTP) lub 443 (HTTPS), co pozwala na łatwe pokonanie zapór sieciowych i serwerów proxy. WebSocket jest idealnym rozwiązaniem dla aplikacji wymagających komunikacji w czasie rzeczywistym, takich jak czaty, gry wieloosobowe, powiadomienia, systemy monitorowania czy aplikacje streamingowe.

Specyfikacja protokołu zawarta jest w dokumencie RFC6455: The WebSocket Protocol. Z formalnego punktu widzenia WebSocket stanowi odrębny, bazujący na TCP, protokół. Z protokołem HTTP wiąże go w zasadzie tylko sam początek zestawiania transmisji, czyli tzw. handshake interpretowany przez serwer jako żądanie "Upgrade". Domyślnie wykorzystywane są porty 80 (dla standardowej transmisji WebSocket) oraz 443 (dla transmisji tunelowanej przez protokół TLS).

Jak działa WebSocket?

1. Zestawianie połączenia (handshake)

- Komunikacja WebSocket zaczyna się od wysłania żądania HTTP z nagłówkiem Upgrade, aby poprosić serwer o przełączenie z protokołu HTTP na WebSocket. Przykład:

```
GET /chat HTTP/1.1
Host: <a href="http://server.example.com">server.example.com</a>
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: x3JJHMbDL1EzLkh9GBhXDw==
Sec-WebSocket-Version: 13
```

- Jeśli serwer obsługuje WebSocket, odpowiada pozytywnie:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: HSmrc0sMlYUkAGmm5OPpG2HaGWk=
```

2. Utrzymanie połączenia

- Po nawiązaniu połączenia, klient i serwer mogą przesyłać dane w dowolnym kierunku bez konieczności wysyłania kolejnych żądań HTTP. Należy pamiętać, że protokół WebSocket udostępnia jedynie dwukierunkowy kanał komunikacyjny, natomiast sam sposób zorganizowania komunikacji leży po stronie programisty - należy zastosować specyficzny dla danej aplikacji, **podprotokół**. Można do tego celu wykorzystać inne znane protokoły lub zaimplementować własny.

3. Zakończenie połączenia

- Połączenie może zostać zakończone przez klienta lub serwer, np. w wyniku normalnego zakończenia pracy aplikacji lub jako konsekwencja wystąpienia błędu.

7. RESTful API

RESTful API (ang. *Representational State Transfer Application Programming Interface*) to rodzaj interfejsu programowania aplikacji (API), który opiera się na zasadach architektury REST (*Representational State Transfer*). REST to styl architektoniczny który definiuje zbiór zasad umożliwiających tworzenie skalowalnych i łatwych w utrzymaniu systemów komunikacji sieciowej. RESTful API to zatem interfejs, który pozwala na komunikację między klientem a serwerem w sposób zgodny z zasadami REST. Zasady te zostały zdefiniowane w kontekście protokołu HTTP i to z nim są najczęściej wykorzystywane. Ich zastosowanie porządkuje i ujednolica sposób wykorzystania różnych elementów protokołu HTTP do realizacji typu CRUD (ang. *Create, Read, Update, Delete*).

Wskazówka

REST został zaproponowany w roku 2000 przez Roya Fieldinga w jego **rozprawie doktorskiej**.
Roy Fielding to współautor **specyfikacji protokołu HTTP/1.0**, współfundator projektu Apache HTTP Server.

Choć założenia stylu REST są wciąż aktualne i stosowane w praktyce, na przestrzeni minionych lat podjęte zostały prace standaryzujące, rezultatem których są m.in. dostępne narzędzia służące do projektowania API. Docelowo warto swoje zainteresowania skierować na OpenAPI Specification (OAS). Przyjrzymy się jednak założeniom przyświecającym stylowi REST, gdyż są one zastosowane również w OAS.

Jak działa RESTful API?

Jako że API w stylu REST bazuje na protokole HTTP, generalnie rzecz biorąc sposób organizacji komunikacji w żądania wysyłane przez klienta do serwera jest taki sam jak dla protokołu HTTP.

- **Klient wysyła żądanie HTTP**:
  - Klient, np. przeglądarka lub aplikacja mobilna, wysyła żądanie do serwera, podając URL zasobu oraz wykorzystując odpowiednią metodę HTTP (GET, POST, PUT, DELETE itp.).
- **Serwer przetwarza żądanie**:
  - Serwer interpretuje żądanie, wykonuje odpowiednią operację na zasobie (np. pobiera dane z bazy danych) i przygotowuje odpowiedź.
- **Serwer zwraca odpowiedź**:
  - Odpowiedź zawiera reprezentację zasobu (np. w formacie JSON), a także kod statusu HTTP wskazujący, czy operacja się powiodła (np. 200 OK, 404 Not Found).

Zasadnicza różnica względem "surowego HTTP" leży więc w uporządkowaniu tej komunikacji i nałożeniu reguł, które pozwalają łatwo zrozumieć sposób działania API.

W kolejnych rozdziałach omówione zostały podstawowe koncepcje definiujące styl architektoniczny REST.

7.1. Koncepcja zasobu

Zasób reprezentuje wszystko, co można identyfikować, przechowywać, manipulować lub przekazywać w systemie. Może to być dowolny element w aplikacji. Pod pojęciem zasobu może kryć się np. użytkownik, produkt, zamówienie, dokument, artykuł czy nawet wynik operacji.
Zasoby są identyfikowane za pomocą unikalnych adresów URL (ang. *Uniform Resource Locator*).

Przykłady zasobów wskazywanych przez adresy URL:

- https://api.serwer.com/**users** – zasób, jakim jest lista użytkowników.
- https://api.serwer.com/**users/1** – zasobem jest konkretny użytkownik o identyfikatorze 1.
- https://api.serwer.com/**products/123** – zasobem jest produkt o identyfikatorze 123.

Zasób jest dostarczany klientowi w określonym formacie, zwanym **reprezentacją**. Najpopularniejszym formatem opisującym zasób jest obecnie JSON, ale spotyka się także XML. Ponadto stosowane są także inne formaty takie jak HTML, tekst czy binarne dane (np. obrazy, filmy, dźwięki), w zależności od wymagań danej aplikacji. Co istotne, format reprezentacji nie musi być tożsamy z formatem przechowywania danych.

Przykład

Przykład zasobu reprezentowanego w formacie JSON:

```
    {
        "id": 1,
        "name": "Jan Kowalski",
        "email": "jan.kowalski@serwer.com"
    }
```

Przykład analogicznego zasobu w formacie XML:

```
<user>
    <id>1</id>
    <name>Jan Kowalski</name>
    <email>jan.kowalski@serwer.com</email>
</user>
```

7.2. Zasady identyfikowania zasobów

Projektując interfejs API w stylu REST dla aplikacji internetowej należy zadbać o właściwe skonstruowanie adresów identyfikujących poszczególne zasoby (ang. *URI – Uniform Resource Identifiers*), gdyż wpływa to na jego spójność i łatwość zrozumienia.

Poniżej zestawiono najważniejsze reguły i dobre praktyki, którymi powinien kierować się twórca aplikacji internetowej podczas projektowania adresów URI dla zasobów w RESTful API.

1. Unikalne i opisowe URI dla zasobów

- Każdy zasób w API powinien mieć jednoznacznie identyfikowalny adres URI.
- Adresy URI powinny być opisowe, aby użytkownicy API mogli łatwo zrozumieć, co jest reprezentowane przez dany adres.
- Przykład:
  - Dla kolekcji użytkowników: `/users`
  - Dla konkretnego użytkownika: `/users/{id}` np. `/users/123`

2. Stosowanie rzeczowników zamiast czasowników

- Poszczególne człony adresów URI powinny być rzeczownikami, które odnoszą się do reprezentowanych zasobów (np. `users`, `products`, `orders`), a nie do akcji (np. `getUser`, `createOrder`).
- Akcje na zasobach (np. tworzenie, odczyt, aktualizacja, usuwanie) są definiowane za pomocą metod protokołu HTTP (np. GET, POST, PUT, DELETE), a nie w samym URI.
- Przykład:
  - Poprawne: `/products`
  - Niepoprawne: `/getProducts`, `/deleteUser`

3. Hierarchiczna struktura URI

- Adresy URI powinny być zorganizowane hierarchicznie, od ogółu do szczegółu, aby odzwierciedlały relacje między zasobami.
- Przykład:
  - Kolekcja użytkowników: `/users`
  - Szczegóły konkretnego użytkownika: `/users/{id}` np. `/users/123`
  - Lista zamówień konkretnego użytkownika: `/users/{id}/orders`

4. Stosowanie liczby mnogiej dla kolekcji

- Kolekcje zasobów powinny być nazwane w liczbie mnogiej, aby jasno wskazywać, że reprezentują zbiór elementów.
- Przykład:
  - Poprawne: `/users`, `/products`, `/orders`
  - Niepoprawne: `/user`, `/product`, `/order` (mogą mylić się z pojedynczym zasobem)

5. Używanie identyfikatorów w URI

- Dostęp do konkretnego zasobu w kolekcji powinien być możliwy za pośrednictwem unikalnego identyfikatora
- Identyfikatory powinny być czytelne i proste (np. liczby lub unikalne ciągi alfanumeryczne)
- Przykład:
  - Konkretne zasoby w kolekcji: `/users/123`, `/products/45`

6. Unikanie wielopoziomowej złożoności URI

- Adresy URI powinny być możliwie krótkie i proste. Głębokie zagnieżdżenie (`/users/123/orders/456/items/789`) powinno być stosowane z rozwagą i tylko wtedy, gdy jest to uzasadnione logicznie.
- Przykład:
  - Poprawne: `/orders/456` (zakładając, że `456` (identyfikator zamówienia) wystarczająco identyfikuje zamówienie, włącznie z powiązaniem go z konkretnym użytkownikiem).
  - Niepoprawne: `/users/123/orders/456` (uznać należy za niepoprawne, jeśli `456` wystarcza do pełnej identyfikacji zamówienia).

7. Wersjonowanie API

- Projektując API warto przewidzieć jego długoterminowy rozwój. W szczególności może się okazać, że w przyszłości niezbędna będzie przebudowa aplikacji i idąca za tym zmiana formatów reprezentacji zasobów. Nie zawsze istnieje możliwość aktualizacji oprogramowania klienckiego, więc dobrze zaprojektowane API powinno przewidywać mechanizm jego wersjonowania. Jedną z możliwości jest umieszczenie umieszczenie identyfikatora wersji w adresie URI.

- Wersjonowanie API powinno może być umieszczane w ścieżce URI lub w nagłówkach, **ale nie w nazwach zasobów**.
- Przykład wersjonowania w URI:
  - Poprawne: `/v1/users`, `/v2/products`
  - Niepoprawne: `/users_v1`

8. Adresy URI zgodne z zasadami konstruowania adresów URL

- Adresy URI powinny być zgodne z zasadami URL-friendly, co oznacza:
  - Używanie małych liter (unikaj wielkich liter)
  - Zamiast spacji stosowanie znaku myślnika (`-`) lub podkreślenia (`_`)
  - Unikanie znaków specjalnych (np. `!@#$%^&*`)
- Przykład:
  - Poprawne: `/user-profiles`, `/order-history`
  - Niepoprawne: `/UserProfiles`, `/order history`

9. Używanie parametrów zapytania dla filtrów, sortowania i paginacji

- Jeśli zasób wymaga dodatkowych informacji, takich jak filtrowanie, sortowanie czy paginacja, te operacje powinny być obsługiwane przez parametry zapytania (query parameters), a nie w samej ścieżce URI.
- Przykład:
  - Paginacja: `/users?page=2&limit=20`
  - Filtrowanie: `/products?category=electronics`
  - Sortowanie: `/products?sort=price&order=desc`

10. Unikanie rozszerzeń plików w URI

- URI powinny być czyste i nie zawierać rozszerzeń plików (np. `.json`, `.xml`), ponieważ format reprezentacji zasobu jest określany przez nagłówki protokołu HTTP (`Accept`), a nie przez adres URI.
- Przykład:
  - Poprawne: `/users`
  - Niepoprawne: `/users.json`, `/users.xml`

11. Odpowiednie użycie kodów odpowiedzi protokołu HTTP

- Adresy URI powinny być zaprojektowane tak, aby pozwalały na wykorzystanie standardowych kodów statusu HTTP (np. `200 OK`, `201 Created`, `404 Not Found`, `500 Internal Server Error`).
- Przykładowe zachowanie:
  - Żądanie `GET /users/123` dla istniejącego użytkownika powinno zwrócić `200 OK` wraz z danymi użytkownika.
  - Jeśli użytkownik o ID `123` nie istnieje, odpowiedź powinna zostać odesłana z kodem `404 Not Found`.

12. Spójność w całym API

- Adresy URI powinny być spójne i jednolite w całym API, niezależnie od zasobu.
- Przykład:
  - Jeśli URI dla użytkowników to `/users/{id}`, to URI dla produktów powinno być w podobnym formacie, np. `/products/{id}`.

13. Obsługa działań specyficznych dla zasobów

- Działania specyficzne dla zasobów, które nie pasują do standardowych metod HTTP (np. akceptowanie zamówienia, resetowanie hasła), można obsługiwać za pomocą pod-ścieżek lub specjalnych zasobów.
- Przykład:
  - Akceptowanie zamówienia: `POST /orders/{id}/accept`
  - Resetowanie hasła użytkownika: `POST /users/{id}/reset-password`

14. Dokumentacja URI

- Wszystkie adresy URI powinny być dokładnie udokumentowane w API, aby użytkownicy wiedzieli, jak z nich korzystać.
- Dokumentacja powinna zawierać:
  - Opis każdego URI
  - Przykłady żądań i odpowiedzi
  - Informacje o obsługiwanych metodach HTTP

Przykłady dobrze zaprojektowanych identyfikatorów zasobów w RESTful API

Metoda HTTP	URI	Opis

Metoda HTTP	URI	Opis
GET	`/users`	Pobierz listę wszystkich użytkowników.
GET	`/users/123`	Pobierz dane użytkownika o ID `123`.
POST	`/users`	Utwórz nowego użytkownika.
PUT	`/users/123`	Zaktualizuj dane użytkownika o ID `123`.
DELETE	`/users/123`	Usuń użytkownika o ID `123`.
GET	`/users/123/orders`	Pobierz zamówienia użytkownika o ID `123`.
POST	`/orders`	Utwórz nowe zamówienie.
GET	`/products?category=books`	Pobierz produkty z kategorii "books".

7.3. Metody HTTP

Operacje na zasobach najczęściej należą do kategorii określanych akronimem CRUD, pochodzącym od określeń typowych operacji:

- Create - utworzenie zasobu,
- Read - odczytanie, pobranie zasobu,
- Update - aktualizacja zasobu,
- Delete - usunięcie zasobu.

API budowane w stylu REST wykorzystuje metody HTTP do realizacji operacji CRUD na zasobach:

- GET - pobranie zasobu,
- POST - utworzenie nowego zasobu,
- PUT - aktualizacja istniejącego zasobu,
- PATCH - częściowa aktualizacja zasobu,
- DELETE - usunięcie zasobu.

Odpowiedzi odsyłane przez serwer opatrzone są zawsze tzw. **kodem statusu** (*HTTP response status code*). Przyjrzyjmy się kodom, które najczęściej są stosowane w projektowaniu interfejsów RESTful API.

1xx – Kody informacyjne

Kody tej grupy rzadko są stosowane, ponieważ w interfejsach RESTful API zazwyczaj nie wymaga się zaawansowanego sterowania przepływem informacji w trakcie obsługi żądania.

2xx – Kody sukcesu

Kody tej grupy wskazują, że żądanie zostało pomyślnie przetworzone przez serwer.

200 OK

- Żądanie zostało pomyślnie przetworzone i serwer zwraca oczekiwane dane.
- Przykład użycia:
  - `GET /users/123` – Zwraca dane użytkownika o ID `123`
  - `PUT /users/123` – Zaktualizowano dane użytkownika

201 Created

- Żądanie zostało przetworzone i zasób został pomyślnie utworzony.
- Przykład użycia:
  - `POST /users` – Utworzono nowego użytkownika
  - Odpowiedź powinna zawierać nagłówek `Location` wskazujący URI nowo utworzonego zasobu (np. `Location: /users/123`)

202 Accepted

- Żądanie zostało zaakceptowane do przetworzenia, ale operacja nie została jeszcze zakończona.
- Przykład użycia:
  - `POST /tasks` – Zadanie zostało przyjęte do przetworzenia, ale serwer jeszcze go nie ukończył.

204 No Content

- Żądanie zostało pomyślnie przetworzone, ale serwer nie zwraca żadnych danych.
- Przykłady użycia:
  - `DELETE /users/123` – Użytkownik został usunięty, w związku z tym nie ma informacji do zwrócenia
  - `PUT /users/123` – Zaktualizowano dane użytkownika, ale nie ma potrzeby zwracania dodatkowych informacji

3xx – Kody przekierowania

Kody te są używane w przypadkach, gdy zasób został przeniesiony lub serwer wymaga dodatkowego działania ze strony klienta.

301 Moved Permanently

- Zasób został trwale przeniesiony do nowego URI
- Przykład użycia:

- `GET /old-resource` – Zwraca kod 301 z nagłówkiem `Location: /new-resource`

302 Found

- Ten kod odpowiedzi oznacza, że URI żądanego zasobu został tymczasowo zmieniony. Dalsze zmiany w URI mogą zostać wprowadzone w przyszłości, więc ten sam URI powinien być używany przez klienta w przyszłych żądaniach.
- Przykłady użycia:
  - Może być użyte do przekierowania na tymczasową lokalizację zasobu.

304 Not Modified

- Zasób nie został zmieniony od ostatniego żądania.
- Przykłady użycia:
  - Nagłówek ten jest używany do celów buforowania. Informuje klienta, że odpowiedź nie została zmodyfikowana, więc klient może nadal korzystać z tej samej buforowanej wersji odpowiedzi. Serwer może nie odsyłać samego zasobu, a przeglądarka natychmiast wykorzysta zasób przechowywany w lokalnej pamięci cache.

4xx – Kody błędów klienta

Kody te wskazują, że przyczyna problemu leży po stronie klienta, np. żądanie zostało nieprawidłowo skonstruowane lub nie podano właściwych danych do autoryzacji.

400 Bad Request

- Żądanie jest niepoprawne lub niezgodne z wymaganiami API.
- Przykłady użycia:
  - `POST /users` – Brak wymaganych danych w ciele żądania
  - `GET /users/abc` – Nieprawidłowy identyfikator (np. `abc` zamiast liczby)

401 Unauthorized

- Klient nie został uwierzytelniony
- Przykład użycia:
  - `GET /users` – Klient nie dostarczył tokena autoryzacyjnego w nagłówku `Authorization`.

403 Forbidden

- Klient nie ma uprawnień do wykonania żądania, nawet jeśli jest uwierzytelniony.
- Przykład użycia:
  - `DELETE /users/123` – Użytkownik zalogowany jako zwykły użytkownik próbuje usunąć innego użytkownika, ale brak mu odpowiednich uprawnień

404 Not Found

- Żądany zasób nie został znaleziony na serwerze.
- Przykłady użycia:
  - `GET /users/999` – Użytkownik o ID `999` nie istnieje.
  - `GET /non-existing-resource` – Zasób nie istnieje.

405 Method Not Allowed

- Metoda HTTP użyta w żądaniu nie jest obsługiwana dla danego zasobu.
- Przykłady użycia:
  - `PUT /users` – Metoda PUT nie jest dozwolona dla kolekcji użytkowników (ale może być dozwolona dla `/users/{id}`).

5xx – Kody błędów serwera

Kody te wskazują, że przyczyna problemu leży po stronie serwera.

500 Internal Server Error

- Ogólny błąd serwera wskazujący, że serwer nie może przetworzyć żądania (bez wskazania bardziej szczegółowej przyczyny).
- Przykład użycia:
  - Wystąpił nieoczekiwany błąd po stronie serwera podczas przetwarzania żądania, np. rzucony został wyjątek, który nie został nigdzie obsłużony.

501 Not Implemented

- Metoda żądania nie jest obsługiwana przez serwer i nie może zostać obsłużona.

- Jedynymi metodami, które serwery muszą obsługiwać (i dlatego nie mogą zwracać tego kodu) są GET i HEAD.
- Przykład użycia:
  - Serwer powinien zwracać ten kod w przypadku, gdy otrzyma żądanie np. typu PUT, którego obsługa nie została zaimplementowana w kontekście danego zasobu.

503 Service Unavailable

- Serwer jest tymczasowo niedostępny (np. z powodu konserwacji lub przeciążenia).
- Przykłady użycia:
  - Serwer API jest wyłączony w celu przeprowadzenia konserwacji.

Wskazówka

Projektując RESTful API należy zadbać, aby to właśnie te kody wykorzystywać do komunikowania np. faktu wystąpienia błędu, a nie poprzez samodzielnie tworzone w tym celu mechanizmy.

Przykład: W sytuacji wystąpienia błędu niedopuszczalne jest wysyłanie kodu **200 OK** a w treści zasobu informacji o błędzie lub wręcz pustej odpowiedzi. Na przykład w sytuacji braku sukcesu autoryzacji, należy odesłać odpowiedź z kodem **401 Unauthorized**.

Pełne zestawienie kodów odpowiedzi protokołu HTTP wraz z ich omówieniem znajdziesz na portalu MDN w artykule: HTTP response status codes.

7.4. Specyfikacja OpenAPI

Specyfikacja OpenAPI (ang. *OpenAPI Specification*, OAS) definiuje standardowy, niezależny od języka, sposób projektowania interfejsów API HTTP, który pozwala zarówno ludziom jak i komputerom odkrywać i rozumieć możliwości usługi bez dostępu do kodu źródłowego, dokumentacji lub poprzez inspekcję ruchu sieciowego. Po prawidłowym zdefiniowaniu konsument może zrozumieć i wchodzić w interakcje ze zdalną usługą przy minimalnej ilości logiki implementacji.

Definicja OpenAPI może być następnie wykorzystana przez narzędzia do generowania dokumentacji do wyświetlania API, narzędzia do generowania kodu do generowania serwerów i klientów w różnych językach programowania, narzędzia testujące i wiele innych przypadków użycia.

Formalna specyfikacja: https://swagger.io/specification/

Zapoznawanie się z OpenAPI Specification warto rozpocząć od artykułów zebranych na poniższych podstronach:

- Swagger Documentation
- API Design